

2. 데이터분석 AIOps

6. 모델 서빙 및 AIOps 구성

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

1일차

Introducing Model Serving	p.6~40
Introducing Model Serving Patterns	p.41~68
Application 내 추론 런타임 (Scikit-learn)	p.69~76
Application 내 추론 런타임 (PyTorch)	p.78~87
Application 내 추론 런타임 (TensorFlow)	p.89~98
Continuous Model Evaluation	p.100~126
Scikit-learn 추론 런타임 실습	p.77
PyTorch 추론 런타임 실습	p.88
TensorFlow 추론 런타임 실습	p.99

2일차

배치 학습 및 서빙	p.127~136
파이프라인 모델 서빙 - 일반	p.137~174
Ensemble Model Serving Pattern	p.176~187
Business Logic Pattern	p.188~208
전용 Serving 도구 - KServe	p.209~230
전용 Serving 도구 - vLLM	p.232~250
파이프라인 모델 서빙 실습	p.175
KServe 실습	p.231
vLLM 실습	p.251

3일차

MLflow	p.252~277
Observability - 일반	p.281~298
LGTM	p.299~333
DevOps	p.335~346

MLOps	p.347~355
LLMOps	p.356~388

MLflow 실습	p.278
LGTM 실습	p.334
LLMOps 실습	p.389
평가 (퀴즈)	

제외 항목

전용 Serving 도구 - Ray Serve	p.391~410
Kubeflow	p.411~434
LangSmith	p.435~448



1일차

아이리스(Iris) 데이터셋을 분석해 모델을 학습한 뒤 pickle로 파일에 저장
그 파일을 다시 불러와 분석(추론)
그 모델을 FastAPI로 감싸서 curl로 직접 호출해 확인

이 전체 과정(학습 → pickle 저장 → 재로드 → FastAPI 서빙 → curl 확인)을
scikit-learn·PyTorch·Keras 세 프레임워크로 각각 연습

2일차

그 "FastAPI + curl" 수준의 서빙을 한 단계 더 끌어올려,
실제 운영 환경에서 쓸 수 있는 서버(오토스케일링·장애 복구·표준 API가 갖춰진 서버)로 배포

1일차가 "내 컴퓨터에서 curl로 확인되는 수준"이었다면,
2일차는 "여러 사람이 안정적으로 쓸 수 있는 실제 서버"다.

문제 1 (그 모델을 만드는 과정을 자동 반복시키기)

04.base-airflow -> 데이터 수집~학습~평가를 DAG로 묶어 자동 스케줄링/트리거

문제 2 (1일차의 "FastAPI+ curl" 서빙을 실제 운영 서버로 끌어올리기 — 3가지 방식)

05.KServe -> Kubernetes 선언적 YAML로 표준화된 배포

06.Ray Serve -> 파이썬 코드로 여러 로직·모델을 직접 조합하는 범용 서빙 프레임워크

07.vLLM -> 생성형 LLM 전용 고성능 추론 서버 (OpenAI API 호환)

05.KServe

이미 학습된 전통 ML 모델을 K8s 표준 규격으로 배포

inferenceservice.yaml의 storageUri가 가리키는, 이미 어딘가에 저장된 sklearn 모델 파일을 그대로 읽어 서빙한다.
YAML 한 장을 선언하면 나머지(오토스케일링, 헬스체크, 표준 REST 엔드포인트)는

KServe가 알아서 맞춰주는 대신, Kubernetes 클러스터가 전제 조건이다.

06.Ray Serve

"여러 로직·모델을 파이썬 코드로 직접 조합하는 범용 프레임워크"

목적은 실제 모델 추론이 아니라, Ray Serve가 여러 개의 독립 배포 단위(Deployment)를 파이썬 코드 레벨에서 `.remote()` 호출로 조합해 하나의 서비스 그래프로 묶어내는 메커니즘 자체를 보여주는 데 있다.
전처리 모델·임베딩 모델·추론 모델 등을 각각의 Deployment로 넣어 파이프라인을 구성한다.

07.vLLM

"생성형 LLM 전용 고성능 추론 서버"

05·06과 달리 분류·회귀 모델이 아니라 텍스트를 생성하는 LLM에 특화되어 있고, OpenAI Chat Completions API와 호환되는 스펙으로 요청을 받으며, Continuous Batching·PagedAttention 같은 LLM 전용 최적화가 내장돼 있다.

그 외

Airflow (파이프라인 오케스트레이션)

Prefect

Airflow보다 나중에 나온 파이썬 네이티브 오케스트레이터.
DAG를 사전 등록하지 않고 일반 파이썬 함수처럼 실행하면서
동적으로 흐름을 구성할 수 있음
태스크 실행 자체보다 "데이터 에셋이 최신인가"에 초점을 맞추는 방식
Airflow Asset 개념과 철학이 비슷함
Airflow와 달리 각 단계를 프로세스가 아니라 K8s Pod 단위로 실행

Dagster

Kubeflow Pipelines / Argo Workflows

KServe (K8s 네이티브 모델 서빙)

Seldon Core

KServe처럼 K8s 네이티브지만 그래프 기반으로 여러 모델·전처리 조합에 강점.
다만 v2부터 라이선스가 Business Source License(BSL)로 바뀌어 프로덕션 사용 시
상용 라이선스가 필요할 수 있음(Apache 2.0인 KServe와의 실질적 차이)
여러 프레임워크(PyTorch/TensorFlow/ONNX) 모델을 한 서버 프로세스에서
동시 서빙, GPU 활용 최적화에 강함
K8s가 필수는 아니고, 파이썬 코드로 모델을 패키징해 로컬·클라우드·K8s
어디든 배포 가능한 개발자 친화적 도구

NVIDIA Triton Inference Server

BentoML

Ray Serve (범용 파이썬 서빙 프레임워크)

BentoML

Ray Serve가 "Ray 클러스터 위 분산 실행"에 강점이면,
BentoML은 "빠른 패키징 후 어디든 배포"에 강점
1일차 방식 그대로.
간단하면 충분하지만 복수 모델 조합·오토스케일링·GPU 스케줄링은 직접 구현해야 함

FastAPI 직접 사용(프레임워크 없이)

vLLM (LLM 전용 추론 서버)

Hugging Face TGI

NVIDIA TensorRT-LLM

HF 생태계 모델 배포에 최적화, 토큰 스트리밍·분산 추론 지원
NVIDIA GPU에 극도로 최적화된 저수준 추론 엔진,
최고 성능이나 NVIDIA 하드웨어 종속
프롬프트·생성 단계 실행 최적화 특화,
에이전트 파이프라인처럼 프롬프트를 반복 재사용하는 구조화된 생성 작업에 강점
개인 컴퓨터·로컬 환경용 경량 실행.
vLLM처럼 다중 사용자 동시 처리에 최적화된 게 아니라 1인 개발/테스트용

SGLang

Ollama / llama.cpp

선수 지식

Container

Docker

애플리케이션과 이를 실행하는 데 필요한 모든 라이브러리·환경설정·의존성(dependencies)을 하나로 묶어 독립된 환경에서 실행되도록 보장해 주는 컨테이너화 플랫폼이다.
개발 환경과 운영 환경의 불일치로 발생하는 "내 컴퓨터에서는 잘 되는데?" 문제를 해결한다.

이번 4개 실습 모두가 "docker compose로 여러 컴포넌트를 띄운다."는 동일한 구조를 쓴다.

04는 Airflow 단일 컨테이너

05는 kind로 가상 K8s 노드 자체가 컨테이너

06/07은 각각 Ray Serve·vLLM 단일 컨테이너

Docker Compose

여러 개의 컨테이너로 구성된 복합 애플리케이션을 정의하고 실행하기 위한 멀티 컨테이너 오케스트레이션 도구다. docker-compose.yml 파일 하나로 다중 컨테이너 환경을 통합 관리한다.

docker compose build

docker compose up(-d는 백그라운드)

docker compose down

Dockerfile을 기준으로 이미지를 빌드

정의된 서비스를 일괄 생성·실행

실행 중인 컨테이너·네트워크·임시 자원을 일괄 정지·삭제

Docker Image vs Docker Container

이미지

컨테이너

소스코드·라이브러리·환경변수·실행명령을 포함한 불변의 Read-Only 템플릿

그 이미지를 기반으로 격리된 프로세스 공간에서 실제로 실행되는 인스턴스

하나의 이미지로부터 상태가 각각 다른 여러 컨테이너를 독립적으로 생성·실행할 수 있다.

예: 04.base-airflow의 Dockerfile은 apache/airflow:3.3.1 이미지에 scikit-learn만 얹은 이미지다.

Volumes를 통한 데이터 지속성

컨테이너 내부 파일 시스템은 컨테이너가 삭제되면 데이터도 사라지는 휘발성(Stateless) 특성을 가진다.

Volume은 호스트 OS의 파일 시스템 공간을 컨테이너 내부 디렉토리나 매핑(Mount)해

컨테이너가 삭제·재시작되어도 데이터를 보존하는 메커니즘이다.

04의 docker-compose.yml이 ./dags, ./plugins, ./data를 볼륨으로 마운트하는 이유다.

컨테이너를 새로 띄워도 DAG 코드와 학습 산출물(model_*.pkl 등)이 남아 있어야 하기 때문이다.

Kubernetes(K8s)

Pod

쿠버네티스에서 배포할 수 있는 가장 작은 컴퓨팅 단위다.

하나 이상의 컨테이너(주로 1개)를 감싸고 있으며, IP 주소와 볼륨 자원을 공유한다.

Deployment

Pod의 배포와 선언적 상태 관리를 담당하는 상위 컨트롤러다.

원하는 Pod 개수(Replicas), 롤링 업데이트, 롤백, 자가 치유를 자동화한다.

KServe의 InferenceService는 Pod를 직접 만들지 않고 Deployment를 생성해 이 기능을 그대로 물려받는다.

Service

Pod 세트에 접근할 수 있는 단일 진입점(가상 IP 및 DNS 이름)을 제공하고 로드 밸런싱을 수행한다.

Pod는 재시작될 때마다 IP가 계속 바뀌므로 안정적인 접근을 위해 Service가 필요하다.

05실습의 port-forward svc/sklearn-iris-predictor가 접근하는 대상이 바로 이 Service다.

Namespace

하나의 클러스터 안을 여러 개의 논리적 공간으로 나누는 개념이다.

같은 이름의 리소스라도 Namespace가 다르면 독립적으로 존재할 수 있다.

05실습의 inferencervice.yaml이 namespace: ml-serving을 명시하고,

정리 단계에서 kubectl delete namespace ml-serving만으로 이 실습에서 만든 자원을 한 번에 정리하는 이유도 여기에 있다.

선언적(Declarative) 설정과 kubectl apply

명령적(Imperative) 방식
선언적 방식

"Pod A를 실행해라", "개수를 3개로 늘려라"처럼 구체적인 동작을 지시
"이 시스템은 Pod 3개와 Service 1개가 있는 상태여야 한다"를 YAML로

kubectl apply -f manifest.yaml은 이 선언적 YAML(최종 상태)을 읽어 현재 클러스터 상태(Current State)와 비교한 뒤 차이만 자동으로 동기화한다.

05실습이 적용 전 --server-side --dry-run=server로 먼저 검증하는 것도 이 메커니즘을 이용한 것이다.

CRD(Custom Resource Definition)와 KServe

CRD는 쿠버네티스 기본 리소스(Pod, Service 등) 외에, 사용자가 정의한 새로운 리소스 타입을 K8s API에 등록해 사용할 수 있도록 확장하는 기능이다.

KServe는 InferenceService라는 CRD를 등록해 두고, 개발자는 K8s 본연의 명령어로 이 개체를 선언해 GPU 할당·오토스케일링·모델 로딩을 통한 관리한다.

kind (Kubernetes in Docker)

Docker 컨테이너 내부를 쿠버네티스 노드로 가상화해, 별도의 클라우드 환경 없이 로컬 Docker 위에서 손쉬게 가상 K8s 클러스터를 생성해주는 개발용 도구다.

05실습은 `kind create cluster --name kserve-base-lab`로 이 실습 전용 가상 클러스터를 만든다.

DAG(Directed Acyclic Graph)

"단일 스크립트를 여러 독립적인 task로 쪼개고, 그 사이의 실행 순서를 화살표로 명시한 것"이며, Airflow는 이 화살표들을 보고 어떤 task를 언제 실행 가능할지를 스스로 판단한다.

실습은 t1_ingest → t2_preprocess → (t3_train_logreg, t4_train_rf 동시 실행) → t5_evaluate_select 순서다.

화살표에 방향이 없다면(무방향 그래프)
만약 순환이 있다면(

t1과 t2 중 어느 쪽이 먼저인지 정할 수 없고,
t1이 실행되려면 t5가 끝나야 하는데 t5는 t1이 끝나야 시작되므로
영원히 아무것도 시작할 수 없는 교착 상태(deadlock)가 된다.

그래서 Airflow는 DAG 정의를 파싱하는 시점에 그래프를 순회하며 순환이 있는지 검사하고, 있으면 그 자리에서 등록 자체를 거부한다.

Directed(방향성)

간선(Edge)에 화살표와 같은 방향이 존재해, 작업 간의 선후 관계(의존성)가 명확히 정의된다.

Acyclic(비순환)

그래프 내부를 따라 이동할 때 자기 자신에게 되돌아오는 순환(Loop/Cycle) 경로가 존재하지 않는 구조다.

04의 dag_invalid_cycle_demo.py가 일부러 t_a >> t_b >> t_a로 순환을 만들어 이 규칙이 설계가 아니라 파싱 단계에서 강제되는 규칙임을 증명한다.

정점(Vertex/Node)와 간선(Edge)

정점
간선

파이프라인 내의 개별 처리 단계(Stage/Task, 예: 수집·전처리·학습)
각 단계 간의 실행 순서 및 데이터 흐름(의존성)

```

[데이터 수집 A] —> [데이터 전처리 B] ———> [모델 학습 C]
                                   |
                                   └——> [데이터 검증 D]

```

무한 루프나 교착 상태(Deadlock)에 빠지지 않고 작업 진행 방향을 보장할 수 있어, Airflow·Kubeflow·Argo Workflows와 같은 MLOps 데이터 파이프라인 오케스트레이션 도구의 핵심 자료구조로 사용된다.

Airflow 아키텍처와 태스크 정의 방식

데이터 수집→전처리→학습→평가 같은 여러 단계를 그냥 쉘 스크립트로 순서대로 실행하면 되지 않을까 싶지만, 실무에서는 특정 단계가 실패했을 때 어디서부터 재시도할지, 정해진 시간마다 자동으로 돌릴지, 단계 사이에 값(정확도 등)을 어떻게 전달할지, 실행 이력을 어떻게 추적할지가 전부 문제가 된다.

Airflow는 이 전체 흐름을 "DAG"라는 하나의 Python 객체로 표현하고, 그 DAG를 실제로 움직이는 여러 프로세스로 역할을 나눠서 처리한다.

- ① DAG Processor가 dags/ 폴더의 .py 파일을 주기적으로 읽어
태스크 목록과 순서(DAG 구조)를 파악해 Metastore에 등록
- ② Scheduler가 schedule 조건(@daily 등)과 Metastore의 마지막 실행 기록을 비교해
"지금 새 DAG Run을 만들어야 하는가"를 판단
- ③ Executor(LocalExecutor)가 그 DAG Run에 속한 태스크들을 실제 프로세스로 실행
- ④ 각 태스크는 끝나면 성공·실패 상태와 XCom 값을 Metastore에 기록
- ⑤ 사용자는 API Server(웹 UI)로 이 전체 과정을 실시간으로 확인한다.

airflow standalone 4개 컴포넌트

04의 docker-compose.yml은 컨테이너 하나 안에서 airflow standalone 명령으로 Scheduler(DAG를 주기적으로 판단해 태스크를 큐에 넣음), Triggerer(비동기 이벤트를 기다리는 트리거 전용 프로세스), DAG Processor(파일을 파싱해 DAG 객체로 변환), API Server(웹 UI)를 한꺼번에 띄운다.

네 개 컴포넌트가 각각 다른 역할을 하는데도 sys.path(임포트 경로)가 서로 다를 수 있다는 것이 아래 "플러그인이 반드시 plugins/에 있어야 하는 이유"의 전제이다.

Metastore와 Executor

Metastore는 DAG 메타데이터·태스크 상태·XCom 값을 저장하는 DB로, 04실습은 기본값인 SQLite(/opt/airflow/airflow.db)를 그대로 쓴다.
Executor는 태스크를 실제로 어디서 실행할지 결정하는데, 기본값인 LocalExecutor는 하나의 머신(여기서는 컨테이너) 안에서 프로세스를 병렬로 띄우는 방식이다.

태스크 정의 3방식 — BashOperator · PythonOperator · @task

BashOperator(예: t1_ingest) 쉘 명령을 그대로 태스크로 등록
PythonOperator(예: t2_preprocess) Python 함수를 태스크로 실행

가장 새로운 방식인 @task 데코레이터(TaskFlow API, 예: dag_asset_pair.py의 write_report_source)는 일반 Python 함수에 데코레이터만 붙여도 Airflow 태스크로 동작하게 만드는, Operator 방식보다 간결한 문법이다.

schedule과 catchup 파라미터

schedule은 DAG가 언제 자동 실행될지를 결정한다.

"@daily"는 매일, None은 수동 트리거만,
이번 04실습에서는 Asset 객체를 넣으면 그 Asset 이벤트가 스케줄이 된다.
catchup=False는 DAG가 처음 활성화될 때 과거 구간을 전부 돌려받지 않고 최근 구간만 실행하게 한다.

XCom(Cross-Communication)

태스크간에 작은 값을 주고받는 Airflow의 내장 메시징 메커니즘이다.
태스크 함수가 값을 return하면 자동으로 Metastore에 XCom으로 저장되고,
다른 태스크가 ti.xcom_pull(task_ids=[...])로 그 값을 읽어온다.

dag_ml_serving_pipeline.py의 evaluate_select(ti) 함수가 두 학습 태스크(t3_train_logreg, t4_train_rf)가 반환한 정확도를 XCom으로 모아오는 것이 실제 예다.

파일이 아니라 DB를 통해 태스크간에 정보를 주고받는다라는 개념이 중요하다.

Airflow 트리거 3방식

시간 기반 schedule(예: @daily)은 구현이 가장 간단하지만 "데이터가 실제로 언제 준비됐는지"와 무관하게 정해진 시각에만 돈다는 한계가 있다.

데이터가 새벽 3시에 이미 준비돼도 정오까지 기다리거나,
반대로 정오에 아직 준비가 안 됐는데 실행돼 빈 데이터로 돌 수도 있다.

이 한계를 이벤트 기반으로 해결하는 두 방식이 Asset과 AssetWatcher다.

Asset(push)은 Airflow 내부의 한 DAG가 "내가 방금 이 데이터를 갱신했다"고 적극적으로 선언하는 방식이고 (예: asset_producer가 report_source.csv를 쓴 뒤 outlets로 선언 → 그 즉시 asset_consumer가 트리거),

AssetWatcher(pull)는 스스로 "내가 갱신했다"고 알려줄 수 없는
Airflow 바깥의 대상을 Airflow가 대신 지켜보는(polling) 방식이다.

Sensor(poke)는 가장 오래되고 단순한 방식으로, 태스크 자체가 원하는 조건이 될 때까지 반복 확인한다.

이때 대기하는 동안 Worker 슬롯을 계속 점유한다는 비용이 있다.

AssetWatcher는 Triggerer라는 별도의 경량 프로세스 안에서 비동기로 돌기 때문에

Worker를 점유하지 않는다는 점이, 실무에서 단순 폴링(Sensor)보다 AssetWatcher를 선호하는 이유다.

Schedule · Asset(push) · AssetWatcher(pull) · Sensor(poke)

이 네 가지는 "DAG가 언제 실행되는가"를 결정하는 서로 다른 방식이며, 04의 5개 DAG 중 4개가 이 비교를 위해 존재한다.

Schedule(시간 기반)

dag_ml_serving_pipeline.py처럼 schedule="@daily"로

cron과 같은 시간표에 따라 자동 실행되는, 가장 전통적인 방식이다.

Asset(push)

태스크가 @task(outlets=[some_asset])로 자신이 특정 Asset("데이터가 갱신했다"는 사실 자체)을 갱신했다고 적극적으로 선언하고, 그 Asset을 schedule=[some_asset]로 구독하는 다른 DAG가 자동으로 트리거되는 방식이다.

dag_asset_pair.py의 asset_producer(report_source.csv를 쓰고 출력을 선언)

→ asset_consumer(그 Asset을 구독)가 이 구조다.

검증 포인트는 consumer의 run_id가 manual__이 아니라 asset_triggered__로 시작하는지다.

AssetWatcher(pull)

Asset이 다른 DAG의 outlets가 아니라,

Airflow 밖의 소스를 커스텀 트리거가 직접 폴링(pull)해서 갱신되는 방식이다.

dag_asset_watcher.py는 AssetWatcher(trigger=DirectoryFilePatternTrigger(...))를 써서,

이 커스텀 트리거가 Triggerer 프로세스 안에서 watched_uploads/ 폴더를 3초마다 감시하다가
새 *.csv 파일이 나타나면 Asset을 갱신시켜 DAG를 트리거한다.

이 커스텀 트리거 클래스가 반드시 dags/가 아니라 plugins/에 있어야 하는데,

Triggerer가 DAG 파일을 재실행하지 않고 DB에 저장된 클래스 경로 문자열만으로 트리거를 재구성하기 때문이다.

이 컨테이너의 sys.path에는 dags/가 없고 plugins/만 있다.

Sensor(poke)

Operator 자체가 특정 상태가 될 때까지 주기적으로 확인(poke)하는 방식이다.

dag_file_sensor.py의 FileSensor는 watched_uploads/trigger.txt가 생길 때까지 poke_interval=3초마다 확인하고,
파일이 없으면 timeout=60초 후 실패한다.

mode="poke"는 대기하는 동안 Worker 슬롯을 계속 점유한다는 특징이 있다.

AssetWatcher는 Triggerer에서 비동기로 동작해 Worker 슬롯을 점유하지 않는다는 것이 차이점이다.

KServe 배포 상세 개념

- ① kubectl apply가 InferenceService 리소스를 K8s API 서버에 등록
- ② KServe 컨트롤러가 이 리소스를 감지하고 modelFormat.name(sklearn)에 맞는 ClusterServingRuntime을 찾음
- ③ 그 Runtime이 정의한 컨테이너 이미지(kserve/sklearnserver)로 Pod를 생성하되, 그 안에서 먼저 storage-initializer(Init Container)를 실행해 storageUri의 모델 파일을 Pod 로컬 볼륨에 내려받음
- ④ 다운로드가 끝나야 kserve-container가 그 모델을 메모리에 로드하고 실제 추론 서버(Uvicorn)를 띄움
- ⑤ K8s Service가 이 Pod 앞에 고정 접속 주소를 만들어주고, 클라이언트는 request.json을 그 주소로 curl 요청
- ⑥ minReplicas~maxReplicas 범위 안에서 오토스케일러가 트래픽에 따라 Pod 개수를 조절한다.

이 여섯 단계 전체가 YAML 파일 하나로 선언되고 K8s가 그 상태를 계속 유지해준다는 것이 KServe의 핵심이며, "로그 확인" 실습에서 storage-initializer 로그와 kserve-container 로그가 순서대로 찍히는 이유도 바로 이 ③→④ 구조 때문이다.

ClusterServingRuntime

설치 시 미리 등록하는 "이 모델 포맷은 이 컨테이너 이미지와 실행 인자로 서빙한다"를 정의해둔 CRD다.

inferenceservice.yaml의 modelFormat.name: sklearn이 kserve-sklearnserver라는 ClusterServingRuntime의 supportedModelFormats와 정확히 일치해야 매칭되며, 이가 실제 사용될 컨테이너 이미지(kserve/sklearnserver)와 실행 인자(--model_dir, --model_name)를 결정한다.

storage-initializer → kserve-container 2단계 기동

InferenceService의 Pod는 단일 컨테이너가 아니다.

먼저 storage-initializer가 초기컨테이너(Init Container)로 실행되어 storageUri에서 모델 아티팩트를 내려받아 Pod 로컬 볼륨에 저장하고, 그것이 끝나야 만 본문인 kserve-container가 그 모델을 로드해 실제 추론 서버를 띄운다.

"로그 확인" 단계에서 Model downloaded in N.NN seconds(storage-initializer)와 Uvicorn running on http://0.0.0.0:8080(kserve-container)을 순차로 보게 되는 이유가 이 2단계 구조 때문이다.

minReplicas/maxReplicas와 resources.requests/limits

minReplicas/maxReplicas	Deployment가 유지할 Pod 개수의 범위를 지정해 트래픽에 따라 오토스케일하게 한다. 이 값을 0으로 두면 Serverless 모드에서 스케일-투-제로도 가능하다.
resources.requests	스케줄링 시 최소 보장 리소스
limits	상한이며 초과 시 CPU는 스로틀링되고 메모리는 OOMKilled(강제 종료)

Ray · 분산 실행 기초

파이썬은 GIL 때문에 한 프로세스 안에서 진짜 병렬 실행이 제한적이고, 여러 대의 컴퓨터로 작업을 나누려면 원래는 소켓 통신과 직렬화를 직접 구현해야 하는 번거로운 일이다. Ray는 이 문제를 "함수나 클래스 앞에 @ray.remote만 붙이면 그 호출이 자동으로 적절한 노드에 전달되어 병렬 실행된다"로 단순화한다.

여기서 한 단계 더 나아가 Ray Serve라는 웹서비스 배포 레이어를 쓴다. Greeter·Shouter·Driver 세 클래스가 각각 하나의 Deployment(독립적으로 스케일되는 배포 단위)이고, Driver가 Ingress deployment로서 외부 HTTP 요청을 가장 먼저 받은 뒤 ServeHandle(greeter_handle, shouter_handle)을 통해 내부적으로 Greeter·Shouter를 마치 로컬 함수처럼 호출한다.

서비스를 여러 개의 독립된 Deployment로 쪼개 배포하면서도, 그 사이의 호출은 매번 HTTP 왕복을 거치지 않고 Ray의 RPC로 처리한다는 것이 Ray Serve 구조의 핵심이다.

Ray

분산 처리 애플리케이션 및 AI 알고리즘 워크로드를 손쉽게 스케일링해 주는 오픈소스 병렬 컴퓨팅 프레임워크다.

Head Node와 Worker Node

Head Node

클러스터 전체를 관리하는 메인 노드로, Global Control Store(GCS)를 통해 전체 노드의 메타데이터와 작업 스케줄링, 자원 상태 모니터링을 총괄한다.

Worker Node

헤드 노드로부터 작업을 할당받아 실제 계산(Tasks, Actors execution)을 수행하고 결과를 반환하는 계산 전용 노드다.

06실습은 컨테이너 하나 안에서 ray start --head로 단일노드 클러스터를 구성한다.

RPC(Remote Procedure Call)와 @ray.remote

RPC는 물리적으로 서로 다른 기기/프로세스에 위치한 함수나 메서드를 마치 같은 메모리 공간 내의 로컬 함수를 호출하는 것처럼 실행할 수 있게 해주는 통신 프로토콜이다. Ray에서는 일반 Python 함수/클래스에 @ray.remote 데코레이터를 붙임으로써, 호출 시 네트워크(RPC)를 통해 클러스터 내의 적절한 노드로 분산 전달되어 병렬 처리된다.

Deployment · ServeHandle · Ingress deployment (Ray Serve 전용 용어)

06실습의 app.py는 @ray.remote보다 한 단계 위인 Ray Serve API를 쓴다.

Deployment

@serve.deployment를 붙인 클래스(예: Greeter, Shouter, Driver) 하나를 의미
한 Deployment가 다른 Deployment를 호출할 때 쓰는 핸들

ServeHandle

예: Driver의 greeter_handle.greet.remote(name)

Ingress deployment

외부 HTTP 요청을 가장 먼저 받는 최상위 Deployment를 가리킨다.

Ray Dashboard

클러스터의 노드·Actor·리소스 사용률을 시각적으로 보여주는 웹 UI다.

06실습은 --dashboard-host=0.0.0.0로 띄워 http://localhost:8275에서 확인할 수 있게 해준다.
기본값 127.0.0.1은 컨테이너 밖에서 접근이 안 된다.

LLM 추론 서빙(Inference Serving) 개념

트랜스포머 기반 LLM은 근본적으로 "지금까지의 토큰들을 보고 다음 토큰 하나를 예측"하는 모델이다.

문장을 만들려면 이 과정을 반복해야 한다.

프롬프트를 넣어 토큰 1개를 생성

→ 그 토큰을 프롬프트 뒤에 이어붙여 다시 모델에 넣어 토큰 2개째를 생성

→ 이렇게 자기 자신이 만든 출력을 다시 입력으로 재사용하기 때문에 "자기회귀(autoregressive)"라고 부른다.

이 특성 때문에 두 가지가 뒤따른다.

토큰이 하나씩 순차적으로만 만들어질 수 있으므로,

다 만들어질 때까지 기다리지 않고 만들어지는 대로 바로 보내주는 SSE 스트리밍이 실용적인 응답 방식이 된다.

매 토큰마다 이전 모든 토큰에 대한 계산을 처음부터 반복하면 너무 느리기 때문에,

이전 계산 결과(Key/Value)를 캐시해 재사용해야 한다.

OpenAI Chat Completions API 형식(messages, max_tokens, stream)으로 요청을 보내고, stream=True일 때 응답이 한 번에 오지 않고 delta(증분) 단위로 여러 번 나뉘어 오는 것이 바로 이 자기회귀 생성 과정을 그대로 클라이언트에 노출한 것이다.

토큰 단위 순차 생성(Autoregressive Decoding)

LLM의 텍스트 생성 방식은 한 번에 전체 문장을 출력하는 것이 아니라, 이전 단계까지 생성된 토큰들을 입력으로 다시 넣어 다음 토큰 하나를 확률적으로 예측하는 자동회귀적(Autoregressive) 생성 구조를 따른다.

Step 1: Input → Token A 생성

Step 2: Input + Token A → Token B 생성

Step 3: Input + Token A + Token B -> Token C 생성

OpenAI Chat Completions API 스펙

LLM 서빙 서비스의 사실상 표준(De facto standard) 인터페이스다.

messages
max_tokens
stream

대화 내역 배열로 역할(role: system/user/assistant)과 내용(content)으로 구분
모델이 생성할 수 있는 최대 토큰 수
참(생성되는 대로 토큰을 스트리밍)/거짓(전체 완료 후 한 번에 반환)을 지정

```
{
  "model": "gpt-4o",
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Hello!"}
  ],
  "max_tokens": 50,
  "stream": true
}
```

HTTP 스트리밍 응답(SSE, Server-Sent Events)

기존 HTTP 통신(Request-Response)은 서버가 응답 생성을 완전히 완료할 때까지 클라이언트가 대기해야 하는 한계가 있다.

SSE는 HTTP 단일 연결을 유지한 채, 서버가 생성하는 데이터를 실시간 이벤트 형태(text/event-stream)로 클라이언트에 지속적으로 푸시(Push)하는 방식이다.

이 덕분에 각 토큰이 만들어지는 즉시 화면에 타이핑되듯 바로 출력될 수 있다.

vLLM 핵심 개념 — Continuous Batching과 KV Cache

이 둘은 vLLM을 vLLM답게 만드는 핵심 개념이며, 본문 07단원의 핵심 이론에서 별도 글자 제목으로 다루는 내용인데, 이 사전 지식에 전혀 없어 새로 추가한다.

구체적인 숫자로 비교하면 차이가 분명해집니다. 요청 A(짧은 답, 토큰 10개)와 요청 B(긴 답, 토큰 200개)가 거의 동시에 들어왔다고 하면, 정적 배치는 둘을 한 배치로 묶어 A가 10번째 토큰에서 끝나도 B가 200번째 토큰을 낼 때까지 그 GPU 슬롯을 비워둔 채 기다립니다(190스텝만큼 낭비). Continuous Batching은 A가 끝나는 즉시 그 슬롯에 대기 중이던 요청 C를 바로 채워 넣으므로, GPU가 노는 시간이 거의 없어지고 초당 처리 가능한 전체 토큰 수(throughput)가 크게 늘어납니다. KV Cache는 이 촘촘한 스케줄링이 가능하도록 만드는 전제 조건이기도 합니다 — 각 요청의 이전 토큰 계산 결과가 이미 캐시돼 있어야, 중간에 새 요청을 끼워 넣어도 그 요청은 자기 캐시만 보고 이어서 계산하면 되기 때문입니다. VLLM_CPU_KVCACHE_SPACE가 클수록 동시에 유지할 수 있는 요청의 KV Cache 수가 늘어나 더 많은 요청을 동시에 처리할 수 있지만, 호스트 메모리를 그만큼 점유한다는 트레이드오프가 있습니다.

정적 배치(Static Batching)의 한계

여러 요청을 묶어(배치) GPU에 한 번에 넣어 처리하는 전통적인 방식은, 배치 안 모든 요청이 끝나야 다음 배치가 시작된다.

한 요청이 짧게 끝나도 같은 배치의 다른 긴 요청이 끝날 때까지 GPU가 놀게 되는 비효율이 발생한다.

Continuous Batching(연속/동적 배치)

vLLM은 고정된 배치 대신, 토큰 단위로 요청을 스케줄링해 하나의 요청이 끝나자마자 바로 그 자리에 대기 중이던 새 요청을 끌어넣는 방식을 쓴다.

이 덕분에 GPU가 쉬는 시간이 거의 없어지고 전체 처리량(throughput)이 크게 증가한다.

client.py의 stream=True로 받는 응답이 다른 요청과 동시에 처리되고 있음을 보여주는 것이 바로 이 기능의 결과다.

KV Cache(Key-Value Cache)와 PagedAttention

트랜스포머 모델은 토큰을 하나씩 생성할 때마다 이전 토큰들의 Attention 계산 결과(Key/Value)를 재사용하기 위해 메모리에 보관하는데, 이것이 KV Cache다.

vLLM의 PagedAttention은 이 KV Cache를 OS의 가상 메모리처럼 고정 크기 블록(page) 단위로 관리해, 메모리 단편화를 줄이고 더 많은 요청을 동시에 처리할 수 있게 한다.

compose.yaml의 VLLM_CPU_KVCACHE_SPACE 환경변수가 이 KV Cache에 예약할 호스트 메모리(GiB 단위)를 지정하는 값이다.

Temperature와 그리디 디코딩(Greedy Decoding)

temperature는 다음 토큰을 골라낼 때의 무작위성 정도를 조절한다.

temperature=0은 항상 확률이 가장 높은 토큰만 선택하는 그리디 디코딩으로, 같은 입력에 항상 같은 출력을 보장해 데모의 재현성을 높인다.

client.py가 명시적으로 temperature=0을 설정하는 이유도, 지정하지 않으면 vLLM의 기본 샘플링(temperature=1.0류)이 적용되어 실행마다 결과가 달라지기 때문이다.

Python 기초

데코레이터(Decorator) 패턴

기존 코드를 수정하지 않고, 함수나 클래스의 앞뒤에 특정 기능을 추가/확장할 수 있게 해주는 문법 요소다.

@task	일반 함수를 Airflow 워크플로우 추적 대상 파이프라인 작업으로 변환
@serve.deployment	일반 Python 클래스를 Ray Serve에서 작동하는 분산 API 서비스 인스턴스로 변경

함수를 인자로 전달하는 패턴(First-Class Object)

Python에서 함수는 일급 객체(First-Class Citizen)다.

함수를 일반 변수처럼 다룰 수 있어 다른 함수의 인자로 전달하거나 반환값으로 사용할 수 있다.

대표적인 내장 예가 map(func, iterable)이고, 사용자 정의 콜백 함수를 PythonOperator(python_callable=preprocess)처럼 다른 함수의 인자로 넘기는 방식으로 Airflow에서도 그대로 쓴다.

개발 환경

Apple Silicon Mac(ARM64) 기준으로 작성되어 있으므로, 인텔 Mac이나 Windows/Linux에서는 이미지 태그(platform: linux/arm64, vllm-openai-cpu:latest-arm64 등)를 해당 아키텍처에 맞게 교체해야 한다.

Docker + Docker Compose 포함	네 실습 모두
kind(Kubernetes in Docker, 최신 안정판)	05.base-kserve: kind create cluster로 로컬 K8s 클러스터 생성
kubecti(K8s 버전과 호환되는 버전)	05.base-kserve
uv(Python 패키지/가상환경 관리자)	07.base-vllm 4단계(uv venv --python 3.11).
Python 3.11(uv venv 안 버전)	07.base-vllm client.py 실행용
Apache Airflow 3.3.1(standalone: SQLite 메타스토어 + LocalExecutor)	04.base-airflow 컨테이너 내부
Ray 2.56.1(rayproject/ray:2.56.1-py311 이미지, Ray Serve 포함)	06.rayservice: 온프레미스 실습과 동일 버전으로 고정
vLLM(vllm/vllm-openai-cpu:latest-arm64, CPU 백엔드 전용 이미지)	07.base-vllm: GPU없이 Docker의 ARM CPU로 추론
openai SDK(openai>=1.60,<3, vLLM이 제공하는 OpenAI 호환 API를 호출)	07.base-vllm client.py

Docker Desktop을 먼저 실행해 둔다.
뒤에 나오는 kind도 내부적으로 Docker 위에서 돈다.

각 실습 폴더로 cd한 뒤 followup.md를 순서대로 따라간다.
매 실습이 "0단계: 이동 + 환경변수 설정"부터 시작한다.

포트 충돌이 난 경우를 대비해 실습별 포트 매핑을 미리 확인해 둔다.
Airflow 웹UI(포트는 compose 파일에 명시),
KServe API(kind 클러스터 내부),
Ray Serve HTTP 8010/Dashboard 8275(호스트측 매핑),
vLLM 8000.

각 실습을 끝낼 때는 docker compose down으로 정리하고, 다음 실습으로 넘어간다.

base-airflow

DAG(Directed Acyclic Graph)

정점(task)을 방향 있는 간선으로 연결하고 어느 경로로도 순환을 이루지 않게 구성해,
파이프라인의 각 단계가 정확히 한 방향으로만 실행되도록 강제한다.
Airflow는 DAG 정의를 파싱하는 단계에서 순환이 있으면 그 자리에서 등록 자체를 거부한다.

Pipeline Stage

파이프라인 그래프의 노드가 되는 작업 단위.
단일 스크립트로 수집→정제→추출→학습·테스트·저장을 몰아넣으면
재실행이나 실패 지점 파악이 어려워지므로,
Airflow는 각 stage를 개별 task(t1, t2 등)로 만들고 t1 >> t2처럼 의존성을 명시한다.

Scheduler / Executor / Worker / Metastore

Scheduler	스케줄에 따라 런을 자동 생성
Executor(이 실습은 LocalExecutor)	실제 task를 어디서 어떻게 실행할지 결정
Metastore(이 실습은 SQLite)	DAG/task/XCom 상태를 영구 저장

Asset(push) / AssetWatcher(pull) / Sensor(poke)

세 가지 모두 "cron 없이 자동 트리거"를 구현하지만 방식이 다르다.

Asset	생산자 DAG가 outlets=[...]로 갱신을 적극적으로 알리는 push 방식
Sensor	소비자 DAG가 조건(파일 존재 여부 등)을 주기적으로 직접 찔러보는(poke) pull 방식

Operator / 팬아웃·팬인

BashOperator는 셸 명령을, PythonOperator는 Python 함수를 그대로 task로 등록한다.

t2 >> [t3, t4]	리스트로 묶이면 해당 task들은 동시에 시작(팬아웃)되고,
[t3, t4] >> t5	다시 모이면 둘 다 끝나야만 다음 단계가 시작(팬인)된다.

DAG

base-airflow/followup.md 파일 순서대로 실습

Apache Airflow 3.3.1을 컨테이너 하나(standalone: SQLite 메타스토어 + LocalExecutor)로 띄우고,
명령을 직접 실행하며 확인한다.

- (1) 복잡한 ML 작업을 6단계로 쪼개 파이프라인 DAG(수집→전처리→학습×2(팬아웃)→평가·선정(팬인)→서빙 스모크 테스트)
- (2) DAG의 "Acyclic" 성질 — 순환이 있는 DAG는 파싱 단계에서 거부됨을 반례로 확인
- (3) Scheduler가 @daily만으로 런을 자동 생성하는 것, Executor/LocalExecutor 구성
- (4) Asset(push)/AssetWatcher(pull)/Sensor(poke) — cron 없이 이벤트·폴링으로 자동 트리거되는 3가지 방식의 차이
- (5) Metastore(SQLite)에 실행 순서와 XCom(학습 정확도)이 그대로 기록되는 것을 sqlite3로 직접 조회

1단계(빌드)	Dockerfile	apache/airflow:3.3.1 + scikit-learn
1~2단계	docker-compose.yml	standalone 컨테이너 1개 포트 8280
		dags/plugins/data/watched_uploads 마운트
4·9단계	dags/dag_ml_serving_pipeline.py	6단계 파이프라인 (Bash/PythonOperator, 팬아웃·팬인)
3단계	dags/dag_invalid_cycle_demo.py	순환 반례 — 파싱에서 거부됨
6단계	dags/dag_asset_pair.py	Asset producer/consumer
8단계	dags/dag_asset_watcher.py	AssetWatcher (폴더 pull 감시)
7단계	dags/dag_file_sensor.py	Sensor (FileSensor 폴링)
8단계	plugins/directory_watcher_trigger.py	AssetWatcher용 커스텀 트리거

Docker Desktop(docker, docker compose)만 있으면 된다.
Python·Airflow·라이브러리는 전부 컨테이너 안에 들어있다.

이 실습의 판정 기준은 run_id의 접두사와 DagRun 상태다.
run_id 접두사가 "누가/무엇이 이 실행을 트리거했는지"를 그대로 말해준다.

4·6·7단계	manual__	사람이 dags trigger로 실행
5단계	scheduled__	Scheduler가 @daily로 자동 생성
6·8단계	asset_triggered__	Asset 이벤트로 자동 트리거

실행마다 달라지는 값

run_id 뒤에 붙는 타임스탬프(__2026-09-05T...)
 asset_triggered의 랜덤 접미사(_GsatwWn6)
 start_date의 시:분:초, poke 횟수
 정확도 소수점 자릿수는 실행할 때마다 다르다.

숫자 자체가 아니라 위 표의 "접두사 패턴"과 "state=success" 만 맞으면 정상이다.
 단, ml_serving_pipeline의 t3(logreg=0.9667) > t4(rf=0.9333) 대소 관계는 시드가 고정돼 항상 같다.

0단계 — 이동

```
cd base-airflow
```

1단계 — 컨테이너 빌드 + 기동

```
docker compose up -d --build
```

```
[+] Running 2/2
```

```
✓ Network base-airflow_default      Created
✓ Container base-airflow-airflow-1   Started
```

2단계 — 기동 완료(healthy)까지 대기

healthy가 될 때까지 상태를 반복 확인 (보통 20~40초)

```
until [ "$(docker compose ps airflow --format '{{.Health}}')" = "healthy" ]; do
  echo " ...대기 중: $(docker compose ps airflow --format '{{.Health}}')"; sleep 3
done
echo "healthy!"
sleep 8                                # dag-processor가 dags/ 폴더를 최초 스캔할 시간
```

```
...대기 중: starting
...대기 중: starting
healthy!
```

3단계 — DAG 목록 + 순환 반례가 파싱에서 거부되는 것 확인 (DAG의 "Acyclic")

정상 DAG 목록 (import error가 있으면 이 명령이 exit 1 이라, || true 로 계속 진행)

```
docker compose exec airflow airflow dags list || true
```

순환을 넣은 반례가 여기 에러로 잡혀야 정상

```
docker compose exec airflow airflow dags list-import-errors || true
```

정상 DAG 5개는 등록되지만, 일부러 t_a >> t_b >> t_a 순환을 넣은 invalid_cycle_demo는 파싱 단계에서 거부되어 실행 목록에 오르지 못하고 import error로만 나타난다.
 순환이 있으면 실행 순서를 정할 수 없기 때문이다.

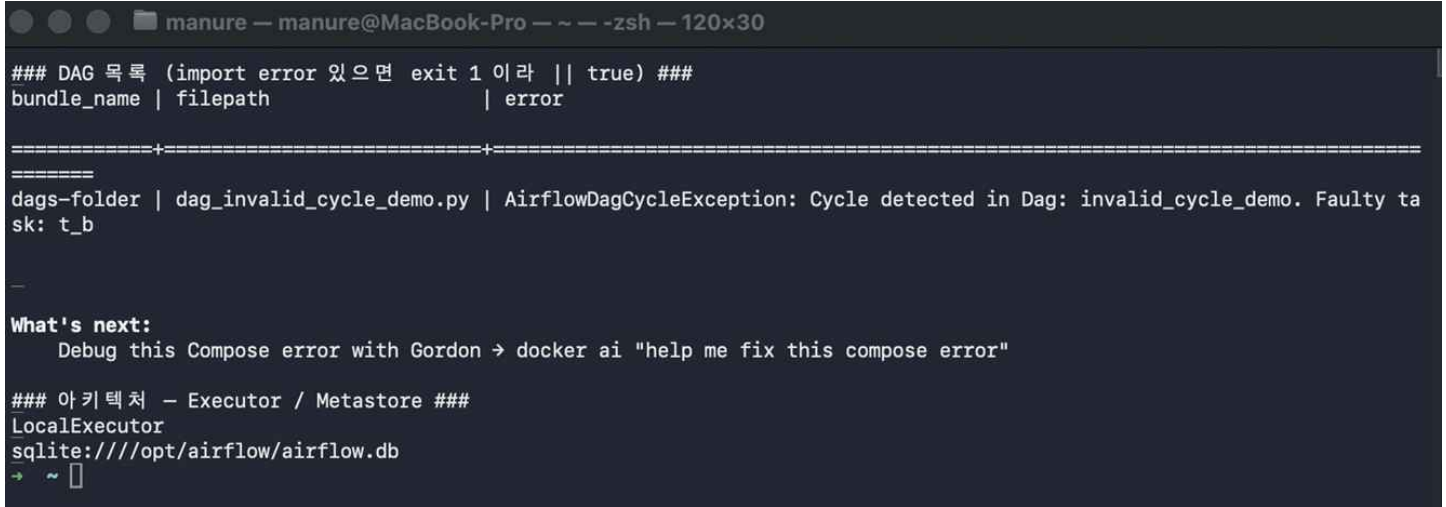
```
--- dags list ---
```

dag_id	fileloc	is_paused
asset_consumer	/opt/airflow/dags/dag_asset_pair.py	True
asset_producer	/opt/airflow/dags/dag_asset_pair.py	True
ml_serving_pipeline	/opt/airflow/dags/dag_ml_serving_pipeline.py	True
process_raw_uploads	/opt/airflow/dags/dag_asset_watcher.py	True
sensor_wait_for_file	/opt/airflow/dags/dag_file_sensor.py	True

이어서 아키텍처 증거도 확인한다.

```
docker compose exec airflow airflow config get-value core executor
```

```
docker compose exec airflow airflow config get-value database sqlalchemy_conn
```



4단계 — 5개 DAG unpause + 6단계 파이프라인 트리거

```
for d in ml_serving_pipeline sensor_wait_for_file asset_producer asset_consumer process_raw_uploads; do
  docker compose exec airflow airflow dags unpause "$d"
done
```

Pipeline Model Serving 6단계 파이프라인 수동 실행

```
docker compose exec airflow airflow dags trigger ml_serving_pipeline -l 2024-03-01T00:00:00+00:00
```

완료될 때까지 상태 확인 (success가 뜰 때까지 반복)

```
docker compose exec airflow airflow dags state ml_serving_pipeline 2024-03-01T00:00:00+00:00
```

t1_ingest(Bash) >> t2_preprocess >> [t3_train_logreg, t4_train_rf] >> t5_evaluate_select >> t6_serve_smoke_test 순서로, 두 모델을 병렬 학습(팬아웃)한 뒤 최고 정확도 모델을 골라(팬인) 서빙 smoke test까지 통과하면 success다.

서빙 스테이지(t6)의 로그를 직접 열어 보면

```
docker compose exec airflow bash -c "grep -h 'W[serveW]' /opt/airflow/logs/dag_id=ml_serving_pipeline/*/task_id=t6_serve_smoke_test/*.log | tail -2"
```

[serve] smoke test — 실제=[0, 1, 0] 예측=[0, 1, 0]

[serve] smoke test 통과 ☒ — 파이프라인 끝단에서 서빙 가능 상태 확인

5단계 — Scheduler 증거: scheduled__ 런이 자동 생성됨

```
docker compose exec airflow airflow dags list-runs ml_serving_pipeline -o plain
```

방금 사람이 만든 manual__ 런 말고, run_id가 scheduled__로 시작하는 런이 함께 보여야 한다.

@daily로 선언했으므로 unpause만 해도 Scheduler가 최근 구간 런을 스스로 만든 것이다.

사람은 이걸 트리거한 적 없다.

ml_serving_pipeline	manual__2026-...	success	← 4단계에서 사람이 트리거
ml_serving_pipeline	scheduled__2026-09-05T00:00:00	success	← Scheduler가 자동 생성 ☒

6단계 — Asset: producer → consumer가 asset_triggered__로 자동 실행

producer가 outlets로 report_source Asset을 갱신한다.

```
docker compose exec airflow airflow dags trigger asset_producer -l 2024-03-01T00:00:00+00:00
```

```
docker compose exec airflow airflow dags state asset_producer 2024-03-01T00:00:00+00:00 # success 대기
```

consumer는 cron 없이 schedule=[report_source] 만 걸려 있다 → 자동으로 돈다

```
docker compose exec airflow airflow dags list-runs asset_consumer -o plain
```

asset_consumer는 사람이 트리거하지 않았는데도 실행됐고, 그 run_id가 asset_triggered__로 시작한다.

Asset 갱신 이벤트가 트리거한 것이다.

```
asset_consumer asset_triggered_2026-... success ← manual이 아니라 asset_triggered ☒
```

7단계 — Sensor: 파일이 나타날 때까지 poke 재시도

먼저 센서 DAG를 트리거. 아직 감시 파일은 없다.

```
docker compose exec airflow airflow dags trigger sensor_wait_for_file -l 2024-03-01T00:00:00+00:00
```

15초쯤 기다렸다가, 감시 대상 파일을 만든다. 그 사이 Sensor는 계속 poke 중

```
sleep 15
```

```
echo "hello sensor" > watched_uploads/trigger.txt
```

파일이 생기면 Sensor가 통과 → success

```
docker compose exec airflow airflow dags state sensor_wait_for_file 2024-03-01T00:00:00+00:00
```

FileSensor가 파일이 없는 동안 poke_interval=3초마다 "아직 없음"을 반복하다가, trigger.txt가 나타나자 통과한다. poke를 여러 번 한 흔적을 로그로 볼 수 있다.

```
docker compose exec airflow bash -c "grep -h 'Poking for file' /opt/airflow/logs/dag_id=sensor_wait_for_file/*/task_id=wait_for_file/*.log"
```

```
success
```

--- poke 로그 (3초 간격으로 여러 번 반복된 뒤 파일이 생기면 통과) ---

```
Poking for file /opt/airflow/watched_uploads/trigger.txt
```

```
Poking for file /opt/airflow/watched_uploads/trigger.txt
```

```
Poking for file /opt/airflow/watched_uploads/trigger.txt
```

```
... (총 6~8회)
```

8단계 — AssetWatcher: 외부 폴더를 직접 폴링(pull)해 자동 트리거

Airflow 바깥에서(사람이 직접) 감시 폴더에 새 csv를 넣는다.

```
printf "id,valueWn1,helloWn2,worldWn" > watched_uploads/new_upload.csv
```

Triggerer 안의 커스텀 트리거가 이 파일을 감지 → process_raw_uploads 자동 실행

```
docker compose exec airflow airflow dags list-runs process_raw_uploads -o plain
```

Asset이 "다른 DAG의 outlets"가 아니라, Triggerer 안에서 폴더를 직접 폴링하는 watcher가 갱신시킨다는 점이 6단계와 다르다. 그래서 파일을 쓰는 쪽이 Airflow 태스크가 아니어도(사람이 그냥 복사해도) 트리거 된다.

Triggerer가 새 파일을 감지한 로그도 직접 확인할 수 있다.

```
docker compose logs airflow | grep "새 파일 감지"
```

```
process_raw_uploads asset_triggered_2026-... success ← asset_triggered ☒
```

[DirectoryFilePatternTrigger] pattern=*.csv 새 파일 감지: ['new_upload.csv'] → TriggerEvent 발생

9단계 — Metastore(SQLite)를 직접 SQL로 열어 실행 기록 확인

```
docker compose exec airflow python - <<'PYEOF'
```

```
import sqlite3
```

```
conn = sqlite3.connect("/opt/airflow/airflow.db")
```

```
print("--- dag_run: run_id 접두사 = 트리거 주체 ---")
```

```
for r in conn.execute("SELECT dag_id, substr(run_id,1,18), run_type, state FROM dag_run ORDER BY dag_id"):
```

```
    print(f" {r[0]:<20} {r[1]:<20} {r[2]:<16} {r[3]}")
```

```
print("Wn--- task_instance: t1>>t2>>[t3,t4]>>t5>>t6 실행 순서 ---")
```

```
for r in conn.execute("""SELECT strftime('%H:%M:%S',start_date), task_id, state FROM task_instance
```

```
WHERE dag_id='ml_serving_pipeline' AND run_id LIKE 'manual_%' ORDER BY start_date"""):
```

```
    print(f" {r[0]} {r[1]:<22} {r[2]}")
```

```
print("Wn--- xcom: 학습 태스크 반환 정확도 ---")
```

```
for r in conn.execute("SELECT task_id, value FROM xcom WHERE dag_id='ml_serving_pipeline' AND task_id LIKE 't%train%'"):
```

```
print(f" {r[0]:<22} {r[1]}")
PYEOF
```

>>로 선언한 TASK 순서가 start_date 순서로 그대로 남고(t3·t4는 같은 초에 시작 = 팬아웃), 학습 TASK가 반환한 정확도까지 XCom 테이블에 저장돼 있다.

```
--- dag_run: run_id 접두사 = 트리거 주체 ---
asset_consumer      asset_triggered_2    asset_triggered    success
asset_producer      manual__2026-09-05    manual            success
ml_serving_pipeline manual__2026-09-05    manual            success
ml_serving_pipeline scheduled__2026-09     scheduled         success
process_raw_uploads asset_triggered_2     asset_triggered    success
sensor_wait_for_file manual__2026-09-05    manual            success

--- task_instance: t1>>t2>>[t3,t4]>>t5>>t6 실행 순서 ---
12:44:18 t1_ingest                success
12:44:20 t2_preprocess            success
12:44:21 t3_train_logreg          success ← t3/t4 같은 초 시작(팬아웃)
12:44:21 t4_train_rf              success
12:44:22 t5_evaluate_select       success ← 둘 다 끝난 뒤(팬인)
12:44:23 t6_serve_smoke_test      success

--- xcom: 학습 TASK 반환 정확도 ---
t3_train_logreg      0.9666666666666667
t4_train_rf          0.9333333333333333
```

[image_followup/follow_02_metastore.png](#)

```
--- dag_run: run_id 접두사 = 트리거 주체 ---
asset_consumer      asset_triggered_2    asset_triggered    success
asset_producer      manual__2026-09-05    manual            success
ml_serving_pipeline manual__2026-09-05    manual            success
ml_serving_pipeline scheduled__2026-09     scheduled         success
process_raw_uploads asset_triggered_2     asset_triggered    success
sensor_wait_for_file manual__2026-09-05    manual            success

--- task_instance: t1>>t2>>[t3,t4]>>t5>>t6 실행 순서 ---
12:44:18 t1_ingest                success
12:44:20 t2_preprocess            success
12:44:21 t3_train_logreg          success
12:44:21 t4_train_rf              success
12:44:22 t5_evaluate_select       success
12:44:23 t6_serve_smoke_test      success

--- xcom: 학습 TASK 반환 정확도 ---
t3_train_logreg      0.9666666666666667
t3_train_logreg      0.9666666666666667
t4_train_rf          0.9333333333333333
t4_train_rf          0.9333333333333333
→ ~ □
```

웹 UI로 보기 + 정리

웹 UI 비밀번호 확인 (아이디: admin) → <http://localhost:8280> 에서 DAG 그래프·로그 열람
[docker compose exec airflow cat simple_auth_manager_passwords.json.generated](#)
[docker compose down -v](#) # 컨테이너·볼륨·네트워크 정리

base-airflow

[base-airflow/Dockerfile](#)

base-airflow 실습 전용 이미지
 # 공식 apache/airflow 3.3.1 위에 ml_serving_pipeline DAG가 쓰는 scikit-learn만 없는다.
[FROM apache/airflow:3.3.1](#)
[RUN pip install --no-cache-dir scikit-learn](#)

[base-airflow/docker-compose.yml](#)

services:

Airflow standalone 컨테이너 하나.
 # 컨테이너 안에서 Scheduler + Triggerer + DAG Processor + API Server(웹 UI)를 전부 띄운다.

```
# Metastore는 기본값인 SQLite (/opt/airflow/airflow.db)
# Executor는 기본값인 LocalExecutor
# "기본 구성 그대로의 Airflow"를 관찰하는 것이 이 실습의 목적이므로 일부러 아무것도 바꾸지 않는다.
airflow:
  build: .
  ports:
    - "8280:8080" # 웹 UI: http://localhost:8280
  environment:
    - AIRFLOW__CORE__LOAD_EXAMPLES=False
  volumes:
    - ./dags:/opt/airflow/dags # DAG 정의 (Python 코드)
    - ./plugins:/opt/airflow/plugins # 커스텀 트리거 — Triggerer도 import 가능해야 함
    - ./data:/opt/airflow/data # 파이프라인 산출물 (모델/피쳐)
    - ./watched_uploads:/opt/airflow/watched_uploads # Sensor·AssetWatcher 감시 대상
# FileSensor의 fs_default 커백션을 만들어준 뒤 standalone 기동
command: >
  bash -c "airflow db migrate &&
            airflow connections create-default-connections &&
            exec airflow standalone"
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8080/api/v2/monitor/health"]
  interval: 5s
  timeout: 5s
  retries: 60
  start_period: 30s
```

DAG

[dags/dag_ml_serving_pipeline.py](#)

ml_serving_pipeline — Pipeline Model Serving을 Airflow DAG로 구현한 메인 실습 DAG.

복잡한 ML 작업(수집→전처리→학습→평가→서빙)을 여러 단계(Pipeline Stage)로 쪼개고, 각 단계를 Airflow 태스크로 정의해 >> 연산자로 실행 순서를 그래프(DAG)로 선언한다.

이 DAG 하나가 다음 개념들을 전부 시연한다.

- Pipeline Model Serving : t6_serve_smoke_test까지 이어지는 다단계 서빙 구조
- DAG : 시작점 1개(t1) → 팬아웃([t3,t4]) → 팬인(t5) → 끝점 1개(t6)
- Pipeline Stage : t1 >> t2 >> [t3, t4] >> t5 >> t6 의 각 노드
- BashOperator : t1_ingest — 셸 명령을 그대로 태스크로
- PythonOperator : t2~t6 — Python 함수를 태스크로
- Many-to-one Dependency : [t3_train_logreg, t4_train_rf] >> t5_evaluate_select
- schedule(@daily) : Scheduler가 매일 자동 실행 (unpause 직후 최근 구간 1회 자동 생성)

```
from __future__ import annotations
import pickle
from datetime import datetime
from pathlib import Path
from airflow.models.dag import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.python import PythonOperator
```

이 실습 폴더(base-airflow/) 기준 데이터 경로 — 모든 스테이지가 파일로 산출물을 주고받는다.
 DATA_DIR = Path(__file__).resolve().parent.parent / "data"

```
def preprocess():
  # t2: Iris 데이터를 로드해 표준화하고, 이후 스테이지가 쓸 features.npz로 저장한다.
  import numpy as np
  from sklearn.datasets import load_iris
  from sklearn.model_selection import train_test_split
  from sklearn.preprocessing import StandardScaler
```

```

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0, stratify=y,
    # stratify=y — 클래스 비율을 유지한 채 train/test를 분할한다는 의미
)
scaler = StandardScaler().fit(X_train)
np.savez(
    DATA_DIR / "features.npz",
    X_train=scaler.transform(X_train).astype("float32"),
    X_test=scaler.transform(X_test).astype("float32"),
    y_train=y_train,
    y_test=y_test,
)
print(f"[preprocess] features.npz 저장 완료 (train={len(y_train)}, test={len(y_test)})")

def train_model(model_name: str):
    # t3/t4: features.npz를 읽어 모델 하나를 학습하고, 정확도를 반환(→ XCom)한다.
    # 반환값이 Metastore(SQLite)의 xcom 테이블에 저장되고, t5가 xcom_pull로 읽는다.
    import numpy as np
    data = np.load(DATA_DIR / "features.npz")
    if model_name == "logreg":
        from sklearn.linear_model import LogisticRegression
        model = LogisticRegression(max_iter=300, random_state=0)
    else:
        from sklearn.ensemble import RandomForestClassifier
        model = RandomForestClassifier(random_state=0)

    model.fit(data["X_train"], data["y_train"])
    acc = float((model.predict(data["X_test"]) == data["y_test"]).mean())

    with open(DATA_DIR / f"model_{model_name}.pkl", "wb") as f:
        pickle.dump(model, f)
    print(f"[train:{model_name}] 테스트 정확도={acc:.4f} -> model_{model_name}.pkl 저장")
    return acc
    # ← XCom으로 Metastore에 저장된다

def evaluate_select(ti):
    # t5: 팬인(many-to-one) 지점 — 두 학습 태스크의 XCom(정확도)을 모아 최고 모델을 고른다.
    import shutil
    accs = ti.xcom_pull(task_ids=["t3_train_logreg", "t4_train_rf"])

    # t3/t4가 반환한 정확도(XCom)를 한 번에 모아옴
    candidates = dict(zip(["logreg", "rf"], accs))
    best = max(candidates, key=candidates.get)
    shutil.copy(DATA_DIR / f"model_{best}.pkl", DATA_DIR / "best_model.pkl")
    print(f"[evaluate] 후보 정확도={candidates} -> best={best} (best_model.pkl로 승격)")
    return best

def serve_smoke_test():
    # t6: 서버 스테이지 — 승격된 best_model.pkl을 로드해 실제 predict가 되는지 확인한다.
    import numpy as np
    with open(DATA_DIR / "best_model.pkl", "rb") as f:
        model = pickle.load(f)
    data = np.load(DATA_DIR / "features.npz")
    sample_X, sample_y = data["X_test"][:3], data["y_test"][:3]
    pred = model.predict(sample_X)
    print(f"[serve] smoke test — 실제={list(sample_y)} 예측={list(pred)}")
    assert list(pred) == list(sample_y), "서빙 smoke test 실패"
    print("[serve] smoke test 통과 ☑ — 파이프라인 끝단에서 서버링 가능 상태 확인")

```

```

with DAG(
    dag_id="ml_serving_pipeline",
    description="Pipeline Model Serving 실습 — 수집→전처리→학습x2→평가→서빙 6단계",
    start_date=datetime(2024, 1, 1),
    schedule="@daily", # Scheduler가 매일 자동 실행 (schedule_interval 개념)
    catchup=False, # 과거 구간 전부가 아니라 최근 구간만
    tags=["base-airflow", "pipeline-model-serving"],
) as dag:
    # t1: BashOperator — 셸 명령을 그대로 Airflow 태스크로 등록
    t1_ingest = BashOperator(
        task_id="t1_ingest",
        bash_command=(
            f"mkdir -p '{DATA_DIR}' && "
            f"date -u +%FT%TZ > '{DATA_DIR}/ingest_marker.txt' && "
            f"echo '[ingest] 원천 데이터 수집 완료 (marker 기록)'"
        ),
    )

    # t2~t6: PythonOperator — Python 함수를 Airflow 태스크로 실행
    t2_preprocess = PythonOperator(task_id="t2_preprocess", python_callable=preprocess)
    t3_train_logreg = PythonOperator(
        task_id="t3_train_logreg", python_callable=train_model, op_kwargs={"model_name": "logreg"}
    )
    t4_train_rf = PythonOperator(
        task_id="t4_train_rf", python_callable=train_model, op_kwargs={"model_name": "rf"}
    )
    t5_evaluate_select = PythonOperator(task_id="t5_evaluate_select", python_callable=evaluate_select)
    t6_serve_smoke_test = PythonOperator(task_id="t6_serve_smoke_test", python_callable=serve_smoke_test)

    # DAG 구조: 시작점 1개 → 팬아웃 → 팬인(many-to-one) → 끝점 1개
    # t3·t4는 리스트로 묶여 병렬(팬아웃)로 동시 시작되고, 둘 다 성공해야만 t5가 시작(팬인)됨
    # 9단계 SQL 조회에서 t3/t4의 start_date가 초 단위로 동일하게 찍히는 것으로 확인 가능
    t1_ingest >> t2_preprocess >> [t3_train_logreg, t4_train_rf] >> t5_evaluate_select >> t6_serve_smoke_test

```

[dags/dag_invalid_cycle_demo.py](#)

invalid_cycle_demo — DAG의 "A(Acyclic, 순환 없음)"가 강제 규칙임을 증명하는 반례 DAG.

`t_a >> t_b >> t_a` 로 일부러 순환을 만들면, 이 파일은 dag-processor의 파싱 단계에서 거부되어 DAG 목록에 아예 등록되지 않는다. airflow dags list-import-errors에 "Cycle detected" 에러로 나타나는 것이 이 실습의 검증 포인트다.

즉 이 파일이 "에러로 남아있는 것"이 정상이다.

순환이 있으면 어떤 태스크를 먼저 실행해야 할지 결정할 수 없으므로, Airflow는 실행 시도조차 하지 않고 파싱에서 막는다.

```

from __future__ import annotations
from datetime import datetime

from airflow.models.dag import DAG
from airflow.providers.standard.operators.bash import BashOperator

with DAG(
    dag_id="invalid_cycle_demo",
    description="순환이 있는 그래프는 DAG가 될 수 없음을 보여주는 반례",
    start_date=datetime(2024, 1, 1),
    schedule=None,
    catchup=False,
    tags=["base-airflow", "dag-반례"],
) as dag:
    t_a = BashOperator(task_id="t_a", bash_command="echo a")
    t_b = BashOperator(task_id="t_b", bash_command="echo b")
    t_a >> t_b >> t_a

```

```
t_a >> t_b
t_b >> t_a
```

← 순환! 파싱 단계에서 "Cycle detected"로 거부된다.

airflow dags list-import-errors로 invalid_cycle_demo가 AirflowDagCycleException으로 등록 거부된 것을 확인하고, 나머지 5개 DAG는 정상 등록되는 것을 대조시키면 "순환 없음"이 설계상의 기대가 아니라 파싱 단계에서 강제되는 규칙임이 명확해진다.

Asset / AssetWatcher / Sensor

위 6단계 파이프라인과 순환 반례 DAG 외에, base-airflow 폴더에는 "데이터가 갱신된 시점"을 기준으로 DAG를 트리거하는 3가지 방식을 보여주는 DAG가 파일 4개(DAG 4개)로 더 들어 있다.

Asset(push)	태스크가 outlets=[...]로 갱신을 선언하면 그 Asset을 schedule=[...]로 구독하는 다른 DAG가 자동 트리거된다.
AssetWatcher(pull)	커스텀 트리거가 Airflow 바깥의 소스(여기서는 디렉터리)를 Triggerer 프로세스 안에서 직접 폴링해 Asset을 갱신시킨다.
Sensor(poke)	Operator 자체가 특정 상태가 될 때까지 주기적으로 확인(poke)한다.

세 방식의 차이를 실제로 눈으로 확인한다.

- ① asset_consumer의 run_id가 manual__이 아니라 asset_triggered__로 시작하는지(Asset push)
- ② watched_uploads/ 폴더에 새 *.csv 파일을 넣었을 때
process_raw_uploads가 poke_interval=3초 이내에 자동 트리거되는지(AssetWatcher pull)
- ③ sensor_wait_for_file 실행 후 trigger.txt가 늦게 생성돼도 wait_for_file 태스크 로그에 poke 시도가
여러 번 찍힌 뒤 성공하는지(Sensor poke).
directory_watcher_trigger.py를 dags/에 두면 Triggerer가 DB에 저장된 클래스 경로 문자열만으로 트리거를 재구성하려다
ModuleNotFoundError로 조용히 실패하므로 반드시 plugins/에 있어야 한다.

dags/dag_asset_pair.py

Asset push 방식(asset_producer → asset_consumer 한 쌍)

cron이 아니라 "데이터가 갱신했다"는 이벤트로 DAG를 잇는다.

- asset_producer 가 report_source.csv 를 쓰고 outlets=[report_asset] 으로 Asset 갱신
- asset_consumer 는 schedule=[report_asset] — cron 없이 Asset 이벤트만으로 자동 트리거

검증 포인트: consumer의 run_id가 manual__ 이 아니라 asset_triggered__ 로 시작한다.

같은 저장소의 MLFLOW/dag3 실습에서 검증된 것과 동일한 패턴.

```
from __future__ import annotations
import csv
from datetime import datetime
from pathlib import Path
from airflow.sdk import DAG, Asset, task
```

```
DATA_DIR = Path(__file__).resolve().parent.parent / "data"
# 두 DAG 사이의 유일한 결합점 — "report_source 데이터가 갱신했다"는 사실 자체를 표현
report_asset = Asset("report_source")
```

```
with DAG(
    dag_id="asset_producer",
    start_date=datetime(2024, 1, 1),
    schedule=None,                    # 실습에선 수동 트리거 (실전이라면 @daily 등)
    catchup=False,
    tags=["base-airflow", "asset"],
):
    @task(outlets=[report_asset])    # 이 태스크 성공 = report_asset 갱신 이벤트 발생
    def write_report_source():
        DATA_DIR.mkdir(exist_ok=True)
        path = DATA_DIR / "report_source.csv"
        with open(path, "w", newline="") as f:
            w = csv.writer(f)
```

```

        w.writerow(["item", "qty"])
        w.writerows([["apple", 3], ["banana", 5], ["cherry", 2]])
        print(f"[producer] {path} 작성 완료 -> outlets로 report_asset 갱신 이벤트 발생")

    write_report_source()

with DAG(
    dag_id="asset_consumer",
    schedule=[report_asset],          # ← cron이 아니라 Asset 이벤트가 스케줄
    catchup=False,
    tags=["base-airflow", "asset"],
):
    @task
    def summarize_report(context):
        # 나를 트리거한 Asset 이벤트 확인. 어떤 데이터 갱신 때문에 실행됐는가
        for asset, events in context["triggering_asset_events"].items():
            print(f"[consumer] 트리거한 Asset: {asset.name} (이벤트 {len(events)}건)")

            with open(DATA_DIR / "report_source.csv") as f:
                rows = list(csv.DictReader(f))
                total = sum(int(r["qty"]) for r in rows)
                print(f"[consumer] report_source.csv {len(rows)}행 읽음 — qty 합계={total}")

    summarize_report()

```

[dags/dag_asset_watcher.py](#)

AssetWatcher pull 방식(process_raw_uploads DAG)

Asset이 "다른 DAG의 outlets"가 아니라 외부 소스를 직접 폴링(pull)해서 갱신되는 구조다.
 AssetWatcher에 연결된 커스텀 트리거(DirectoryFilePatternTrigger)가
 Triggerer 프로세스 안에서 watched_uploads/ 폴더를 3초마다 감시하다가,
 새 *.csv 파일이 나타나면 TriggerEvent → raw_uploads Asset 갱신 → 이 DAG 자동 트리거.

asset_producer/consumer(push 방식 — 태스크의 outlets가 Asset을 갱신)와 대비되는 pull 방식이며,
 파일을 쓰는 쪽이 Airflow 바깥(사람이 cp해도)이어도 동작한다.
 같은 저장소의 ASSETWATCHER(AIRFLOW ASSET WATCHER)/directory_watcher 실습에서 검증된 패턴.

```

from __future__ import annotations
from pathlib import Path
from airflow.sdk import DAG, Asset, AssetWatcher, task
# plugins/directory_watcher_trigger.py — dags/가 아니라 plugins/에 있는 이유는 그 파일 상단 주석 참고
# Triggerer의 sys.path 문제를 실제로 겪고 옮겼다.
from directory_watcher_trigger import DirectoryFilePatternTrigger

WATCHED_DIR = str(Path(__file__).resolve().parent.parent / "watched_uploads")

# 이 Asset은 outlets가 아니라 watcher(트리거)가 직접 갱신시킨다.
raw_uploads_asset = Asset(
    name="raw_uploads",
    watchers=[
        AssetWatcher(
            name="raw_uploads_watcher",
            trigger=DirectoryFilePatternTrigger(
                directory=WATCHED_DIR, pattern="*.csv", poke_interval=3.0
            ),
        ),
    ],
)

```

```

with DAG(
    dag_id="process_raw_uploads",
    schedule=raw_uploads_asset, # watcher가 갱신시키는 Asset을 구독
    catchup=False,
    tags=["base-airflow", "asset-watcher"],
):
    @task
    def print_triggering_files(context):
        """이 DAG를 트리거한 Asset 이벤트(=트리거가 감지한 파일 목록)를 출력한다."""
        for asset, events in context["triggering_asset_events"].items():
            for event in events:
                print(f"[watcher-consumer] Asset '{asset.name}' 갱신 — 감지된 파일: {event.extra}")

    print_triggering_files()

```

plugins/directory_watcher_trigger.py

AssetWatcher에 연결되는 커스텀 트리거 클래스

반드시 dags/가 아니라 plugins/에 있어야 하는 이유는 파일 상단 docstring에 원인이 그대로 남아 있다.

같은 저장소의 검증된 구현(ASSETWATCHER(AIRFLOW ASSET WATCHER)/directory_watcher/dags/directory_watcher_trigger.py — apache-airflow==3.3.1의 공유 스트림 프로토콜 사용)을 그대로 가져왔다.

이 파일이 plugins/ 안에 있어야 하는 이유(이 Docker 실습에서 실제로 겪은 문제)
 Triggerer는 DAG 파일을 재실행하지 않고 DB에 저장된 클래스 경로 문자열
 ("directory_watcher_trigger.DirectoryFilePatternTrigger")만으로 트리거를 재구성한다.

처음에 이 파일을 dags/에 뒀더니 dag-processor는 정상 파싱했지만 Triggerer 로그에
 Trigger failed to load code ... ModuleNotFoundError("No module named 'directory_watcher_trigger'")가 찍히며
 watcher 등록이 조용히 실패했다.

이 컨테이너 환경에서 Triggerer의 sys.path에는 dags/가 없기 때문이다.
 plugins/는 Airflow의 모든 컴포넌트(Scheduler·Triggerer·dag-processor·Worker)가
 sys.path에 포함시키는 폴더라 여기로 옮겨 해결했다.
 원본 실습 주석의 "실전 배포에서는 plugins/나 별도 패키지로 두는 게 정석"이라는 지적이 실제로 맞았던 셈이다.

```

from __future__ import annotations
import asyncio
import fnmatch
import os
from collections.abc import AsyncIterator, Hashable
from typing import Any
from airflow.triggers.base import BaseEventTrigger, TriggerEvent

class DirectoryFilePatternTrigger(BaseEventTrigger):
    # directory 안에서 pattern에 매칭되는 파일이 "새로" 나타나는지 감시하는 트리거.
    def __init__(self, *, directory: str, pattern: str, poke_interval: float = 5.0):
        super().__init__()
        self.directory = directory
        self.pattern = pattern
        self.poke_interval = poke_interval

    def serialize(self) -> tuple[str, dict[str, Any]]:
        return (
            f"{self.__class__.__module__}.{self.__class__.__qualname__}",
            {"directory": self.directory, "pattern": self.pattern, "poke_interval": self.poke_interval},
        )

    def shared_stream_key(self) -> Hashable | None:
        # 같은 (directory, poke_interval) 조합이면 폴더 리스팅 자체를 공유한다.

```

```

        return ("directory-scan", self.directory, self.poke_interval)

    @classmethod
    async def open_shared_stream(cls, kwargs: dict[str, Any]) -> AsyncIterator[dict[str, Any]]:
        directory = kwargs["directory"]
        poke_interval = kwargs["poke_interval"]
        while True:
            names = os.listdir(directory) if os.path.isdir(directory) else []
            yield {"directory": directory, "names": names}
            await asyncio.sleep(poke_interval)

    async def filter_shared_stream(self, shared_stream: AsyncIterator[dict[str, Any]]) -> AsyncIterator[TriggerEvent]:
        seen: set[str] | None = None
        async for snapshot in shared_stream:
            current = {f for f in snapshot["names"] if fnmatch.fnmatch(f, self.pattern)}
            if seen is None:
                # 감시 시작 시점에 이미 있던 파일은 "새 파일"로 치지 않는다.
                self.log.info(
                    "[DirectoryFilePatternTrigger] pattern=%s 감시 시작 — 시작 시점 기존 파일 %d개는 무시: %s",
                    self.pattern, len(current), sorted(current),
                )
                seen = current
                continue
            new_files = current - seen
            if new_files:
                self.log.info(
                    "[DirectoryFilePatternTrigger] pattern=%s 새 파일 감지: %s → TriggerEvent 발생",
                    self.pattern, sorted(new_files),
                )
                yield TriggerEvent({"directory": self.directory, "files": sorted(new_files)})
            return
        seen = current

    async def run(self) -> AsyncIterator[TriggerEvent]:
        # shared_stream_key를 안 쓰는 실행 경로용 폴백(독립 검증용)
        stream = self.open_shared_stream({"directory": self.directory, "poke_interval": self.poke_interval})
        async for event in self.filter_shared_stream(stream):
            yield event

```

[dags/dag_file_sensor.py](#)

Sensor poke 방식(sensor_wait_for_file DAG)

Sensor는 "특정 상태가 될 때까지 주기적으로 폴링(poke)하는 Operator의 한 종류"다.
FileSensor가 watched_uploads/trigger.txt 가 나타날 때까지 poke_interval=3초마다 확인하고,
파일이 생기면 그제서야 다음 태스크(process_file)로 넘어간다.

run.sh가 DAG 트리거 후 15초 뒤에 파일을 생성하므로,
태스크 로그에 poke 시도가 여러 번 찍힌 뒤 성공하는 것("아직 없음 → 재시도 → 감지")을 실제로 볼 수 있다.
같은 저장소의 SENSOR(AIRFLOW SENSOR)/base 실습에서 검증된 패턴

```

from __future__ import annotations
from datetime import datetime
from pathlib import Path
from airflow.models.dag import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.sensors.filesystem import FileSensor

```

filepath가 절대경로면 fs_default 커넥션의 basepath와 무관하게 이 경로를 가리킨다.
WATCH_FILE = str(Path(__file__).resolve().parent.parent / "watched_uploads" / "trigger.txt")

```

with DAG(
    dag_id="sensor_wait_for_file",
    description="Sensor 실습 — trigger.txt가 나타날 때까지 3초마다 poke",
    start_date=datetime(2024, 1, 1),
    schedule=None,
    catchup=False,
    tags=["base-airflow", "sensor"],
) as dag:
    wait_for_file = FileSensor(
        task_id="wait_for_file",
        fs_conn_id="fs_default",
        filepath=WATCH_FILE,
        poke_interval=3,          # 3초마다 존재 여부 확인(poke)
        timeout=60,              # 60초 안에 안 나타나면 실패
        mode="poke",             # poke 모드 — 대기하는 동안 Worker 슬롯을 점유
    )

    process_file = BashOperator(
        task_id="process_file",
        bash_command=f"echo '[process] 감지된 파일 내용:' && cat '{WATCH_FILE}'",
    )

    # Sensor가 게이트 역할 — 파일이 생겨야만 후속 처리가 실행된다.
    wait_for_file >> process_file

```

base-kserve

kind로 로컬 Kubernetes 클러스터를 띄우고 KServe를 설치한 뒤, InferenceService YAML 하나를 kubectl apply해 sklearn 모델이 실제 Pod로 구체화되고 V1 프로토콜로 추론 응답을 주는 과정을 끝단까지 직접 확인한다.

KServe는 "K8s에 모델을 서빙하려면 Deployment·Service·Ingress·HPA·GPU 스케줄링을 매번 직접 조합해야 하는" 문제를, 모델 배포를 하나의 선언(YAML)로 추상화해 해결하는 Kubernetes 위의 모델 배포·라우팅·확장 플랫폼이다.

KServe Controller(Control Plane)	InferenceService/ServingRuntime/ClusterServingRuntime 등 CRD를 watch·reconcile해 실제 Deployment/Service로 구체화
ClusterServingRuntime(Control Plane, CRD)	모델 포맷별(sklearn, Triton, vLLM 등) 기본 서빙 컨테이너 이미지·실행 인자를 미리 등록해둔 템플릿
InferenceService(Control Plane, CRD)	사용자가 선언하는 최상위 리소스.
Predictor(Data Plane)	필수 predictor와 선택적 transformer/explainer를 하나의 스펙으로 묶음 실제 추론을 수행하는 컴포넌트. storage-initializer(init container)가 모델 아티팩트를 내려받은 뒤 서빙 컨테이너가 그 볼륨을 마운트해 기동
Transformer(Data Plane, 선택)	Predictor 앞단에서 요청/응답을 전후처리(예: 이미지 디코딩)하는 별도 컨테이너
Explainer(Data Plane, 선택)	예측 결과에 대한 설명(SHAP 등)을 생성하는 별도 경로

실행 흐름은

"kubectl apply

- KServe Controller가 InferenceService CR 생성을 watch
- ClusterServingRuntime에 매칭된 Pod 스펙을 구체화해 생성
- storage-initializer가 모델 아티팩트를 내려받음
- kserve-container 기동·readiness 성공
- InferenceService 상태가 READY=True로 갱신"이다.

즉 사용자는 Pod를 직접 만들지 않고 "원하는 상태"만 선언하며, 나머지는 Controller가 대행한다.

[base-kserve/followup.md](#)

전용 kind 클러스터를 새로 만들어 KServe를 설치하고, sklearn 모델 하나를 실제로 배포·추론까지 검증한 뒤 클러스터를 깨끗이 정리하는 과정을, 명령을 한 줄씩 직접 쳐보면서 따라간다.

inferenceservice.yaml	배포할 InferenceService CR	
request.json	sklearn-iris 모델(storageUri: gs://kfserving-examples/...) 선언 V1 predict 요청 바디(Iris 샘플 2개)	4단계 7단계

docker(구동 중이어야 함), kind, kubectl, helm, curl.
이 문서는 docker 29.5.3, kind 0.32.0, kubectl 1.36, helm 4.2.2(macOS)로 실제 검증했다.

5단계(Ready 대기)	READY 컬럼이 True
6단계(로그 확인)	storage-initializer가 Model downloaded in N.NN seconds.을 출력 kserve-container가 Uvicorn running on http://0.0.0.0:8080을 출력
7단계(추론 요청)	{"predictions": [1,1]} + HTTP_STATUS:200
8단계(정리)	kind get clusters에서 이 실습의 클러스터 이름이 더 이상 안 보임

값이 아니라 패턴을 봐야 하는 항목
Model downloaded in N.NN seconds.의 숫자(다운로드 소요 시간), kubectl get inferenceservice의 AGE 컬럼,
Pod 이름 뒤의 해시(예: sklearn-iris-predictor-5c778b98c9-qtsfc)는 실행할 때마다 달라진다.

숫자 자체가 아니라 "다운로드가 성공했는가"."몇 초 안팎인가".Pod 이름 패턴이 sklearn-iris-predictor-*인가"만 확인하면 된다.
반면 {"predictions": [1,1]}은 모델 아티팩트가 고정돼 있어 매번 동일하게 나와야 정상이다.

0단계 — 이동

cd base-kserve

1단계 — 전용 kind 클러스터 생성

kind create cluster --name kserve-base-lab

```
Creating cluster "kserve-base-lab" ...
• Ensuring node image (kindest/node:v1.36.1)  ...
✓ Ensuring node image (kindest/node:v1.36.1) 
• Preparing nodes  ...
✓ Preparing nodes 
• Writing configuration  ...
✓ Writing configuration 
• Starting control-plane  ...
✓ Starting control-plane 
• Installing CNI  ...
✓ Installing CNI 
• Installing StorageClass  ...
✓ Installing StorageClass 
```

Set kubectl context to "kind-kserve-base-lab"
You can now use your cluster with:
kubectl cluster-info --context kind-kserve-base-lab
Have a nice day!

다른 실습에서 만든 클러스터와 이름이 겹치지 않는지 미리 확인하고 싶다면

kind get clusters

2단계 — KServe 설치 스크립트 다운로드

```
KSERVE_VERSION=v0.18.0
curl -sL "https://github.com/kserve/kserve/releases/download/${KSERVE_VERSION}/kserve-standard-mode-full-install-with-manifests.sh" \
  -o "/tmp/kserve-standard-install-${KSERVE_VERSION}.sh"
chmod +x "/tmp/kserve-standard-install-${KSERVE_VERSION}.sh"
```

별도 출력 없음(다운로드만 수행).
성공 여부는 파일 크기로 확인한다.

```
ls -la "/tmp/kserve-standard-install-${KSERVE_VERSION}.sh"
```

```
-rwxr-xr-x 1 manure wheel 1692781 Sep  3 15:10 /tmp/kserve-standard-install-v0.18.0.sh
```

3단계 — cert-manager + KServe 설치 (가장 오래 걸리는 단계, 1~3분)

```
bash "/tmp/kserve-standard-install-${KSERVE_VERSION}.sh"
```

실제로 나온 출력을 순서대로 발췌 — 총 90여 줄 중 각 구간의 핵심만

```
[INFO] Installing Helm v3.16.3 for darwin/arm64...
[SUCCESS] Successfully installed Helm v3.16.3 to ...
[INFO] Installing Kustomize v5.8.1 for darwin/arm64...
[INFO] Installing yq v4.52.1 for darwin/arm64...
[INFO] Adding cert-manager Helm repository...
[INFO] Installing cert-manager v1.17.0...
...
[SUCCESS] Successfully installed cert-manager v1.17.0 via Helm
pod/cert-manager-556d7b4d8c-qw48w condition met
pod/cert-manager-cainjector-676796597b-jcwb7 condition met
pod/cert-manager-webhook-7c65796f99-h2tjv condition met
[SUCCESS] cert-manager is ready!
[INFO] Installing KServe CRDs...
customresourcedefinition.apiextensions.k8s.io/inferenceservices.serving.kserve.io serverside-applied
...
[INFO] Installing KServe core components...
deployment.apps/kserve-controller-manager serverside-applied
...
[INFO] Adding deployment mode update: Standard
[INFO] ✓ deploy.defaultDeploymentMode = "Standard"
[SUCCESS] Deployment 'kserve-controller-manager' in namespace 'kserve' is available!
[SUCCESS] KServe is ready!
[INFO] Installing ClusterServingRuntimes...
clusterservingruntime.serving.kserve.io/kserve-sklearnserver serverside-applied
clusterservingruntime.serving.kserve.io/kserve-tritonserver serverside-applied
... (총 11종 ClusterServingRuntime 등록)
=====
☑ Installation completed successfully!
=====
```

맨 마지막 줄이 ☑ Installation completed successfully!인지,
그리고 clusterservingruntime.../kserve-sklearnserver가 등록 목록에 있는지
7단계에서 이 런타임이 자동 매칭돼 sklearn 모델을 서빙한다.

4단계 — 네임스페이스 생성 + InferenceService 적용

```
kubectl create namespace ml-serving
kubectl apply --server-side --dry-run=server -f inferenceservice.yaml
kubectl apply -f inferenceservice.yaml
```

```
namespace/ml-serving created
inferenceservice.serving.kserve.io/sklearn-iris serverside-applied (server dry run)
inferenceservice.serving.kserve.io/sklearn-iris created
```

5단계 — Ready 대기 + 상태 확인

```
kubectl -n ml-serving wait --for=condition=Ready inferenceservice/sklearn-iris --timeout=5m
kubectl -n ml-serving get inferenceservice sklearn-iris
```

```
inferenceservice.serving.kserve.io/sklearn-iris condition met
```

NAME	URL	READY	PREV	LATEST	PREVROLLED	OUTREVISION
LATEST	READY	REVISION	AGE			

sklearn-iris
36s

http://sklearn-iris-ml-serving.example.com

True

READY가 True가 아니면 다음 단계로 넘어가지 말고

kubectl -n ml-serving describe inferencesservice sklearn-iris로 원인을 먼저 확인한다.

6단계 — storage-initializer / kserve-container 로그 확인

```
POD=$(kubectl -n ml-serving get pods -l serving.kserve.io/inferencesservice=sklearn-iris -o jsonpath='{.items[0].metadata.name}')  
echo "$POD"
```

```
kubectl -n ml-serving logs "$POD" -c storage-initializer
```

```
sklearn-iris-predictor-5c778b98c9-qtsfc  
--- storage-initializer ---  
... Initializing, args: (src_uri, dest_path): [('gs://kfserving-examples/models/sklearn/1.0/model', '/mnt/models')]  
... Copying contents of gs://kfserving-examples/models/sklearn/1.0/model to local  
... Downloading: /mnt/models/model.joblib  
... Successfully copied gs://kfserving-examples/models/sklearn/1.0/model to /mnt/models  
... Model downloaded in 10.47547508700518 seconds.
```

```
kubectl -n ml-serving logs "$POD" -c kserve-container
```

```
--- kserve-container ---  
.../sklearn/base.py:376: InconsistentVersionWarning: Trying to unpickle estimator SVC from version 1.0.1 when using version 1.5.2. ...  
... Registering model: sklearn-iris  
... Starting uvicorn with 1 workers  
... Starting gRPC server on [::]:8081  
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

InconsistentVersionWarning은 학습 당시 scikit-learn 버전(1.0.1)과 서빙 컨테이너의 scikit-learn 버전(1.5.2)이 달라서 나오는 경고이지 에러가 아니다.

아래 7단계 추론은 정상적으로 성공한다.

Pod 이름 뒤 해시와 다운로드 초 단위 숫자는 실행마다 달라진다(위 "무엇을 확인해야 하는가" 참고).

7단계 — port-forward + 실제 추론 요청

```
kubectl -n ml-serving port-forward svc/sklearn-iris-predictor 8090:80 & sleep 4
```

```
curl http://localhost:8090/v1/models/sklearn-iris:predict ₩  
-H "Host: sklearn-iris.ml-serving.example.com" ₩  
-H "Content-Type: application/json" ₩  
-d @request.json
```

```
Forwarding from 127.0.0.1:8090 -> 8080  
Forwarding from [::1]:8090 -> 8080  
{"predictions": [1, 1]}  
HTTP_STATUS: 200
```

![kubectl get inferencesservice로 READY=True를 확인하고,

이어서 curl로 실제 추론 요청을 보내 {"predictions": [1, 1]}과 HTTP_STATUS: 200을 받는 실제 macOS 터미널 화면]

[image_followup/follow_01_ready_and_predict.png](#)

```

➔ ~ bash /private/tmp/claude-502/~Users-manure--work-re-scala/33233df6-817b-4621-b588-328dcaca1699/scratchpad/fu_captur
e1.sh
$ kubectl -n ml-serving get inferencesservice sklearn-iris
NAME                                URL                                READY  PREV  LATEST  PREVROLLOUTREVISION  LATESTREADYR
EVISION  AGE
sklearn-iris  http://sklearn-iris-ml-serving.example.com  True
84s

$ curl http://localhost:8090/v1/models/sklearn-iris:predict -H "Host: sklearn-iris.ml-serving.example.com" -H "Content-Type: application/json" -d @request.json
{"predictions":[1,1]}
HTTP_STATUS:200
➔ ~ 

```

주의

-d @request.json처럼 파일로 바디를 보낼 때 -H "Content-Type: application/json"을 반드시 붙여야 한다.
이 헤더 없이 보내면 curl이 기본값(application/x-www-form-urlencoded)으로 보내서
서버가 입력을 숫자 배열이 아닌 스칼라로 잘못 해석해 500 에러가 난다.

요청을 다 보냈으면 port-forward를 종료한다.
다음 단계로 넘어가기 전 필수)

```
lsof -ti:8090 | xargs kill -9
```

8단계 — 정리 (네임스페이스 + kind 클러스터 삭제)

```
kubectl delete namespace ml-serving --wait=true
kind delete cluster --name kserve-base-lab
```

```
namespace "ml-serving" deleted
Deleting cluster "kserve-base-lab" ...
Deleted nodes: ["kserve-base-lab-control-plane"]
```

정리가 끝났으면 이 실습의 클러스터만 없어지고 다른 클러스터·컨테이너는 그대로인지 확인한다.

```
kind get clusters
docker ps --format "{{.Names}}"
```

![kind get clusters와 docker ps로, 방금 만든 클러스터는 목록에서 사라지고
이 실습과 무관한 다른 클러스터(예시 화면의 k8sbible)는 그대로 남아있는 것을 확인하는 실제 macOS 터미널 화면]

[image_followup/follow_02_cleanup_verified.png](#)

```

$ kind get clusters
k8sbible

$ docker ps --format "{{.Names}}"
k8sbible-control-plane

# kserve-base-lab이 목록에서 사라졌고, 다른 실습(k8sbible)은 그대로 남아있다
➔ ~ 

```

[inferencesservice.yaml](#)

```

apiVersion: serving.kserve.io/v1beta1      # KServe 자체 API 그룹 — K8s CRD 컨벤션 위에 KServe가 독자 설계
kind: InferenceService                     # 사용자가 선언하는 최상위 리소스
                                           # predictor(필수) + transformer/explainer(선택)를 하나의 스펙으로 묶음

metadata:
  name: sklearn-iris                     # 배포 이름
                                           # 추론 엔드포인트 경로에도 그대로 쓰인다 (/v1/models/sklearn-iris:predict)

  namespace: ml-serving

  annotations:
    serving.kserve.io/deploymentMode: Standard # Standard 모드
                                           # Knative/Istio/Gateway API 없이 일반 K8s Deployment+ Service로 배포

spec:
  predictor:
                                           # 필수 하위 컴포넌트
                                           # 실제 추론을 수행하는 Pod (storage-initializer → kserve-container 2단계로 기동)

```

```

minReplicas: 1                                # 최소 replica 수 (Serverless 모드라면 0으로 두서 스케일-투-제로도 가능)
maxReplicas: 3                                # 최대 replica 수 — 이 범위 안에서 트래픽에 따라 오토스케일
model:
  modelFormat:
    name: sklearn                             # 모델 포맷 선언
                                              # 이 값으로 ClusterServingRuntime(kserve-sklearnserver)이 자동 매칭됨
  storageUri: gs://kfserving-examples/models/sklearn/1.0/model
                                              # 모델 아티팩트 위치(pull 전용)
                                              # storage-initializer가 여기서 내려받아 Pod 로컬 볼륨에 저장

resources:
  requests:                                   # Pod 스케줄링 시 최소 보장 리소스
    cpu: 100m
    memory: 256Mi
  limits:                                     # Pod가 쓸 수 있는 리소스 상한 — 초과 시 CPU는 스로틀링, 메모리는 OOMKilled
    cpu: "1"
    memory: 1Gi
request.json:
{"instances":[[6.8,2.8,4.8,1.4],[6.0,3.4,4.5,1.6]]}

```

modelFormat.name

sklearn 값이 KServe 설치 시 자동 등록된 ClusterServingRuntime(kserve-sklearnserver)의 supportedModelFormats와 정확히 일치해야 매칭되며, 이 매칭이 실제로 띄는 이미지(kserve/sklearnserver:v0.11.0)와 실행 인자(--model_dir, --model_name)를 결정

rayserve

Greeter(인사말 생성) → Shouter(대문자 강조)를 ServeHandle로 있는 Driver 구조의 Ray Serve 앱을 Docker 컨테이너 하나로 띄운다.
호스트에는 Python도 Ray도 설치하지 않고, 이미지 안에 전부 포함되어 있다.
실제 HTTP 추론을 호출해 결과를 확인하고, 응답에 실린 hostname 값이 실제 컨테이너 ID와 일치하는지 대조해 "정말 이 컨테이너 안에서 서빙되고 있다"는 것을 직접 증명한 뒤, serve status와 Ray Dashboard로 3개 deployment의 상태까지 확인한다.

실제 app.py docstring은 "4가지 배포 방법(온프레미스/Docker/RayCluster CRD/RayService CRD/순수 Deployment)에서 동일하게 재사용하는 Ray Serve 애플리케이션"이라 명시하고 "1_onprem_Rayserve/app.py와 동일한 코드"라고 교차 참조한다.

이 실습은 그 중 Docker 배포 방식(1b) 하나만 쓰며, "같은 앱을 컨테이너 안에서 serve run으로 띄운다"는 점만 온프레미스 버전과 다르다.

Ray Cluster / Head Node / Worker Node	ray start --head로 기동되는 헤드 프로세스가 GCS(전역 상태 관리)와 Raylet(노드별 액터 기동·배치)을 관리한다. Dashboard·HTTP Proxy·ServeController가 모두 이 위에서 뜬다.
ServeController / HTTPProxyActor	ServeController는 배포를 관리하는 컨트롤러(HTTP 프로시 기동, replica 수 조정, 헬스체크). HTTPProxyActor는 고정 주소(127.0.0.1:8000)로 요청을 받아 Ingress deployment(Driver)로 라우팅한다.
Deployment / ServeHandle / Ingress deployment	@serve.deployment로 감싼 클래스 하나하나가 독립 배포 단위다. (Greeter/Shouter/Driver). ServeHandle은 한 Deployment가 다른 Deployment를 원격 호출할 때 쓰는 핸들러, Driver가 생성자에서 greeter_handle·shouter_handle을 받아 .remote()로 호출한다. Ingress deployment는 HTTP 요청을 직접 받는 최상위 Deployment다.

1b_docker_RayServe/followup.md

Docker로 Ray Serve 앱 띄우고 컨테이너 서빙 확인하기

Ray Serve 애플리케이션(Greeter → Shouter를 ServeHandle로 잇는 Driver)을 Docker 컨테이너 하나로 띄워, 호스트에서 실제 HTTP 추론을 해보고 "정말 컨테이너 안에서 서빙되는지"까지 한 줄씩 직접 확인한다.

호스트에는 Python도 Ray도 설치하지 않는다.
전부 이미지 안에 들어있다.

1단계(이미지에 복사됨) app.py

1단계 serve_config.yaml

1단계(빌드) Dockerfile

1~5단계 docker-compose.yml

Ray Serve 앱

@serve.deployment 3개(Greeter/Shouter/Driver)와 ServeHandle 체인

serve run 대상 config

http_options.host: 0.0.0.0(컨테이너 밖 접속용) + app 정의

rayproject/ray:2.56.1-py311 + 위 두 파일

컨테이너 1개 — 포트 8010→8000, 8275→8265, shm_size 2gb, python 헬스체크

docker(구동 중이어야 함), docker compose, curl, python3.

호스트에 Ray/Python 패키지 설치 필요 없다.

이 문서는 rayproject/ray:2.56.1-py311, Docker Compose(macOS)로 실제 검증했다.

컨테이너 STATUS

greeting / shouted

hostname 필드

serve status 3개 deployment

Up ... (healthy)

안녕하세요, Manure님!

안녕하세요, MANURE님!!!!

컨테이너 ID와 같음

⚠ 값 자체는 매 실행마다 다름 — "컨테이너 ID와 일치하는가"만 본다.

전부 HEALTHY, RUNNING: 1

☑ 고정(항상 healthy여야 정상)

☑ 고정(모델 로직이 결정적)

☑ 고정

실행마다 달라지는 값

hostname(=컨테이너 ID), proxy의 해시 ID, last_deployed_time_s, CREATED/Up N seconds는 실행할 때마다 다르다.

숫자 자체가 아니라 "응답 hostname == 실제 컨테이너 ID"라는 일치 관계와

상태값(healthy / HEALTHY / RUNNING) 만 맞으면 정상이다.

0단계 — 이동

cd 1b_docker_RayServe

1단계 — 이미지 빌드 + 컨테이너 기동

docker compose up -d --build

rayproject/ray:2.56.1-py311 위에 app.py·serve_config.yaml을 얹은 이미지를 빌드하고,

컨테이너 안에서 ray start --head(Dashboard를 0.0.0.0에 바인딩) → serve run serve_config.yaml이 실행된다.

[+] Running 2/2

✓ Network 1b_docker_rayserve_default Created

✓ Container 1b_docker_rayserve-rayserve-1 Started

2단계 — healthy가 될 때까지 대기

헬스체크가 통과(Serve 엔드포인트가 실제 200)할 때까지 반복 확인 — 보통 15~30초

until ["\$(docker compose ps rayserve --format '{{.Health}}')" = "healthy"]; do

echo " ...대기 중: \$(docker compose ps rayserve --format '{{.Health}}')"; sleep 3

done

echo "healthy!"

...대기 중: starting

...대기 중: starting

healthy!

3단계 — 호스트에서 실제 추론 + 컨테이너 서빙 증명

컨테이너 상태 확인

docker compose ps

추론 요청 (호스트 8010 → 컨테이너 8000) — Greeter가 인사말을 만들고 Shouter가 강조

curl -s "http://127.0.0.1:8010/?name=Manure" | python3 -m json.tool --no-ensure-ascii

응답 hostname이 실제 컨테이너 ID와 같은지 확인 (= 이 컨테이너가 서빙 주체)

```
CID=$(docker compose ps -q rayserve)
echo "실제 컨테이너 ID(12): ${CID:0:12}"
echo "응답의 hostname      : $(curl -s 'http://127.0.0.1:8010/?name=probe' | python3 -c 'import sys,json; print(json.load(sys.stdin)["hostname"])]')"
```

greeting(Greeter deployment)이 만든 인사말을 shouted(Shouter deployment)가 대문자+!!!로 다시 가공했다.
Driver가 두 deployment를 ServeHandle로 순서대로 원격 호출한 결과다.
그리고 응답의 hostname이 실제 컨테이너 ID와 앞 12자리까지 같으면, 호스트가 아니라 이 컨테이너 안에서 서빙됐음이 증명된다.

```
NAME                                ... STATUS                                PORTS
1b_docker_rayserve-...-1          ... Up 48 seconds (healthy) 0.0.0.0:8010->8000/tcp, 0.0.0.0:8275->8265/tcp
```

```
{
  "greeting": "안녕하세요, Manure님!",
  "shouted": "안녕하세요, MANURE님!!!!",
  "served_by_method": "1b_Docker_Ray컨테이너",
  "hostname": "d3bbca033ad1"
}
```

실제 컨테이너 ID(12): d3bbca033ad1

응답의 hostname : d3bbca033ad1 ← 두 값이 같으면 ☒

![컨테이너가 Up (healthy) 상태이고, curl 추론 응답이 greeting/shouted/served_by_method/hostname 필드로 나오며,
실제 컨테이너 ID와 응답 hostname이 d3bbca033ad1로 일치하는 실제 macOS 터미널 화면

[image_followup/follow_01_predict_hostname.png](#)

```
### 컨테이너 상태 ###
NAME                                IMAGE                                COMMAND                                SERVICE  CREATED      STAT
US                                  PORTS
1b_docker_rayserve-rayserve-1      1b_docker_rayserve-rayserve        "bash -c 'ray start ..."             rayserve  48 seconds ago Up 48
seconds (healthy) 0.0.0.0:8010->8000/tcp, [::]:8010->8000/tcp, 0.0.0.0:8275->8265/tcp, [::]:8275->8265/tcp

### 추론 요청 (Greeter -> Shouter ServeHandle 체인) ###
{
  "greeting": "안녕하세요, Manure님!",
  "shouted": "안녕하세요, MANURE님!!!!",
  "served_by_method": "1b_Docker_Ray컨테이너",
  "hostname": "d3bbca033ad1"
}

### hostname 필드 = 컨테이너 ID 확인 ###
실제 컨테이너 ID(12): d3bbca033ad1
응답의 hostname      : d3bbca033ad1
→ ~
```

4단계 — serve status로 3개 deployment 상태 확인

`docker compose exec rayserve serve status`

Greeter / Shouter / Driver 세 deployment가 각각 HEALTHY, RUNNING: 1이면
앱 전체(greeter_app)가 RUNNING 상태로 정상 서빙 중이라는 뜻이다.

proxies:

0464348...afb5: HEALTHY

applications:

greeter_app:

status: RUNNING

deployments:

Greeter: {status: HEALTHY, replica_states: {RUNNING: 1}}

Shouter: {status: HEALTHY, replica_states: {RUNNING: 1}}

Driver: {status: HEALTHY, replica_states: {RUNNING: 1}}

![serve status 출력 — proxies HEALTHY, greeter_app status RUNNING, 그 아래 Greeter/Shouter/Driver 세 deployment가
각각 status HEALTHY와 replica_states RUNNING 1로 표시된 실제 macOS 터미널 화면

[image_followup/follow_02_serve_status.png](#)

```
proxies:
  04643480513333f57ba76edd15bef86dd8146b845612f797fe18afb5: HEALTHY

applications:
  greeter_app:
    status: RUNNING
    message: ''
    last_deployed_time_s: 1788614126.3858757
    deployments:
      Greeter:
        status: HEALTHY
        status_trigger: CONFIG_UPDATE_COMPLETED
        replica_states:
          RUNNING: 1
        message: ''
      Shouter:
        status: HEALTHY
        status_trigger: CONFIG_UPDATE_COMPLETED
        replica_states:
          RUNNING: 1
        message: ''
      Driver:
        status: HEALTHY
        status_trigger: CONFIG_UPDATE_COMPLETED
        replica_states:
          RUNNING: 1
        message: ''

target_capacity: null
```

5단계 — Ray Dashboard 열어보기 (선택)

open <http://localhost:8275/#/serve>

컨테이너의 Ray Dashboard(8265)가 호스트 8275로 매핑돼 있다.

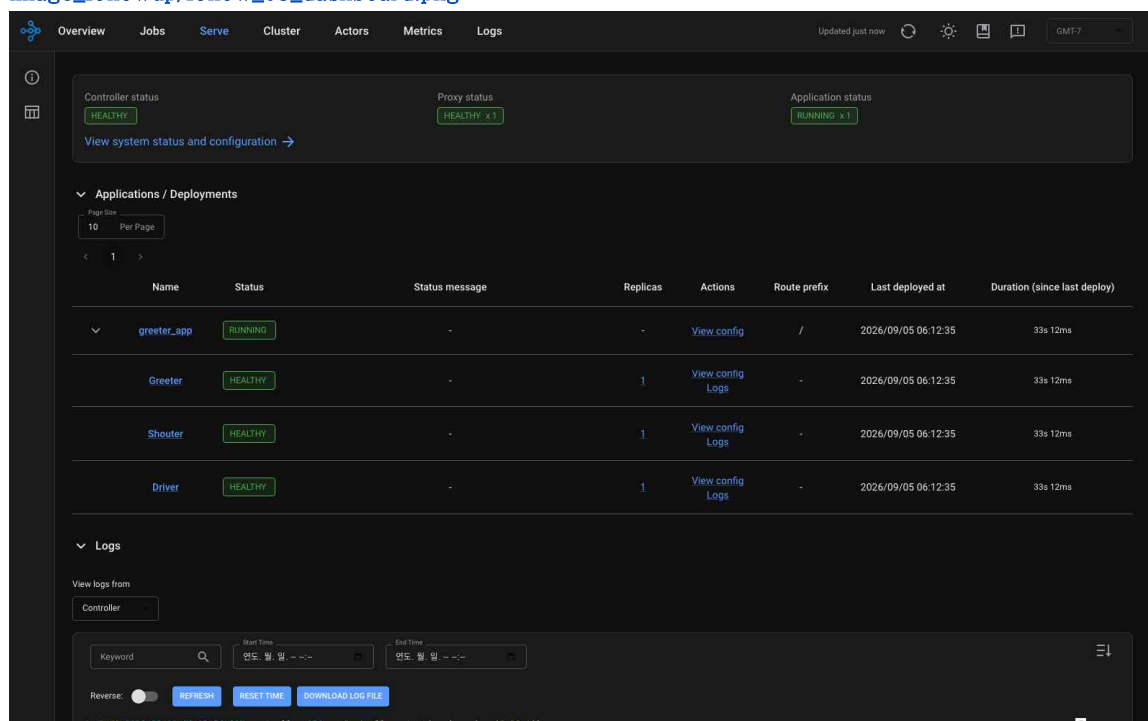
Serve 탭에서 Controller/Proxy/Application 상태와 3개 deployment를 시각적으로 확인할 수 있다.

ray start --head --dashboard-host=0.0.0.0으로 띄웠기 때문에 컨테이너 밖에서 열린다.

기본값이면 컨테이너 안에서만 보인다.

![Ray Dashboard의 Serve 탭 — Controller status HEALTHY, Proxy status HEALTHY x1, Application status RUNNING x1, Applications/Deployments 표에 greeter_app(RUNNING)과 Greeter/Shouter/Driver 세 deployment가 각각 HEALTHY·Replicas 1로 표시된 화면

[image_followup/follow_03_dashboard.png](#)



6단계 — 정리

`docker compose down`

`app.py`

```
# 공통 실습 앱
# 4가지 배포 방법(온프레미스 / Docker / RayCluster CRD / RayService CRD / 순수 Deployment)에서
# 동일하게 재사용하는 Ray Serve 애플리케이션.
```

RAY SERVE.md의 "4-1. 핵심 구성 요소" 3가지를 그대로 실습한다.

- Deployment: Greeter, Shouter, Driver 각각 `@serve.deployment`
- ServeHandle: Driver가 greeter_handle/shouter_handle을 생성자로 받아 `.remote()`로 호출
- Ingress deployment: Driver가 HTTP 요청을 받는 최상위 deployment

`1_onprem_Rayserve/app.py`와 동일한 코드

이 실습은 "같은 앱을 컨테이너 안에서 `serve run`으로 띄운다"는 점만 다르다.

호스트 `venv`가 아니라 Docker 이미지.

```
import os
from starlette.requests import Request
from ray import serve
@serve.deployment
class Greeter:
    # 서버 deployment #1 — 이름을 받아 인사말을 만든다.
    def greet(self, name: str) -> str:
        return f"안녕하세요, {name}님!"

@serve.deployment
class Shouter:
    # 서버 deployment #2 — 문장을 대문자+강조로 바꿔준다.
    def shout(self, text: str) -> str:
        return text.upper() + "!!!"

@serve.deployment
class Driver:
    # Ingress deployment — HTTP 요청을 받아 ServeHandle로 두 서버 deployment를
    # 순서대로 원격 호출(Greeter -> Shouter)한 뒤 결과를 합쳐 반환한다.
    def __init__(self, greeter_handle, shouter_handle):
        self._greeter = greeter_handle
        self._shouter = shouter_handle

    async def __call__(self, request: Request) -> dict:
        name = request.query_params.get("name", "World")
        greeting = await self._greeter.greet.remote(name)
        shouted = await self._shouter.shout.remote(greeting)
        return {"greeting": greeting, "shouted": shouted, "hostname": os.uname().nodename}

greeter = Greeter.bind()
shouter = Shouter.bind()
app = Driver.bind(greeter, shouter)
```

`Dockerfile`

```
# 공식 Ray 이미지(ray[serve] 사전 설치) 위에 실습 앱만 얹는다.
# 온프레미스 실습(1_onprem_Rayserve)이 검증한 ray 2.56.1과 동일 버전으로 고정.
FROM rayproject/ray:2.56.1-py311
WORKDIR /serve
COPY app.py serve_config.yaml ./
```

`docker-compose.yml`

services:

```
# Ray Serve 단일 컨테이너
# 컨테이너 안에서 serve run app:app이 Ray를 자동 기동(serve run은 실행 중인
# Ray가 없으면 로컬 head를 알아서 띄운다)하고 app.py의 Driver를 8000 포트로 서빙한다.
# 온프레미스(1_onprem)가 호스트 venv + ray start --head를 직접 했던 것과 달리,
# 여기서는 그 전부가 이미지 하나 안에 캡슐화돼 docker compose up만으로 끝난다.
```

rayserve:

```
build: .
# Ray object store는 /dev/shm을 쓴다 — 기본값 64MB로는 경고가 띄우려 넘네히 준다.
shm_size: "2gb"
ports:
  - "8010:8000"          # Serve HTTP 엔드포인트 → http://localhost:8010
  - "8275:8265"          # Ray Dashboard → http://localhost:8275
# 1) ray head를 dashboard-host=0.0.0.0으로 먼저 띄운다 — 이래야 Ray Dashboard(8265)를
# 컨테이너 밖(호스트)에서 열 수 있다(기본값 127.0.0.1은 컨테이너 내부에서만 접속됨).
# 2) 그 위에서 serve run을 포그라운드(--blocking 기본)로 실행해 컨테이너를 살려둔다.
# (serve HTTP의 0.0.0.0 바인딩은 serve_config.yaml의 http_options.host가 담당)
```

command: >

```
bash -c "ray start --head --dashboard-host=0.0.0.0 --port=6379 &&
exec serve run serve_config.yaml"
```

healthcheck:

```
# Serve 엔드포인트가 실제 200을 줄 때까지 기다린다(Dashboard가 아니라 앱 자체).
# rayproject 이미지엔 curl이 없어서, 이미지에 있는 python으로 확인한다.
```

```
test: ["CMD", "python", "-c", "import urllib.request,sys; sys.exit(0) if
```

```
urllib.request.urlopen('http://localhost:8000/?name=probe',timeout=3).status==200 else 1)"]
```

```
interval: 5s
```

```
timeout: 5s
```

```
retries: 40
```

```
start_period: 40s
```

serve_config.yaml

Ray Serve config — serve run serve_config.yaml로 띄운다.

컨테이너 밖(호스트)에서 접속하려면 프로시가 0.0.0.0에 바인딩돼야 한다.

최신 Ray의 serve run에는 --host 플래그가 없어서(제거됨), HTTP 바인딩 host/port는 이 config의 http_options로 지정한다.

http_options:

```
host: 0.0.0.0
```

```
port: 8000
```

applications:

```
- name: greeter_app
```

```
import_path: app:app # app.py의 app 객체 (Driver.bind(...))
```

```
route_prefix: /
```

base-vllm

vLLM은 생성형 LLM 추론에 특화된 오픈소스 모델 서버로, LLM은 토큰을 하나씩 순차 생성하고 요청마다 길이가 달라 전통적인 고정 배치 서빙으로는 GPU가 자주 노는 문제를, continuous batching과 Paged KV Cache로 GPU를 상시 가동하면서 요청별로 독립된 SSE 스트리밍 응답을 돌려준다.

OpenAI API Server

요청을 받는 진입점. API key·schema·max_tokens 검사,

요청별 결과를 JSON/SSE chunk로 직렬화

Tokenizer

프롬프트(messages + chat template)를 token IDs로 변환

Scheduler(Continuous Batch Scheduler)

활성 시퀀스 상태를 유지하며 매 decode iteration마다 완료 요청은 빼고

대기 요청은 즉시 투입해 실행 batch를 재구성

Paged KV Cache

요청별 KV 캐시 블록을 예약·갱신·반환

페이지(block) 단위로 관리해 메모리를 효율적으로 사용

Compute Worker(GPU/CPU/Metal)

실제 prefill·decode 연산을 수행

이 실습은 Docker ARM CPU 백엔드로 실제 검증


```
fi
sleep 1
done
```

docker compose up -d가 ARM CPU용 vLLM 컨테이너를 띄운다.
/v1/models가 200을 줄 때까지(=모델 로딩 완료) 폴링한다.

```
[+] Running 2/2
 ✓ Network base-vllm_default          Created
 ✓ Container base-vllm-vllm-cpu-1     Started
준비 완료 (112초)
```

2단계 — /v1/models 로 서빙 중인 모델 확인

```
curl -s -H "Authorization: Bearer ${VLLM_API_KEY}" "http://127.0.0.1:8000/v1/models" \W
| python3 -c 'import json,sys; d=json.load(sys.stdin)["data"][0]; print("id      :", d["id"]); print("root      :", d["root"]);
print("max_len :", d["max_model_len"])'
```

요청한 대로 Qwen/Qwen2.5-0.5B-Instruct를 assistant라는 이름으로, context 길이 1024로 서빙 중임을 확인한다.

```
id      : assistant
root    : Qwen/Qwen2.5-0.5B-Instruct
max_len : 1024
```

![컨테이너가 Up 상태(vllm/vllm-openai-cpu:latest-arm64, 포트 8000)이고, /v1/models 조회 결과 id=assistant, root=Qwen/Qwen2.5-0.5B-Instruct, max_len=1024가 출력된 실제 macOS 터미널 화면]

[image_followup/follow_01_server_models.png](#)

```
### 컨테이너 상태 ###
NAME                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
base-vllm-vllm-cpu-1 vllm/vllm-openai-cpu:latest-arm64  "vllm serve Qwen/Qwe..."           vllm-cpu   2 minutes ago  Up 2 minu
tes    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp

### /v1/models - 서빙 중인 모델 확인 ###
id      : assistant
root    : Qwen/Qwen2.5-0.5B-Instruct
max_len : 1024
→ ~ □
```

3단계 — curl 비-스트리밍 chat completion (temperature=0)

```
curl -s -X POST "http://127.0.0.1:8000/v1/chat/completions" \W
-H "Authorization: Bearer ${VLLM_API_KEY}" -H "Content-Type: application/json" \W
-d      '{"model":"assistant","messages":[{"role":"user","content":"모델 서빙을 한 문장으로 설명해
줘."}], "max_tokens":64,"temperature":0}' \W
| python3 -c 'import json,sys; d=json.load(sys.stdin); print(d["choices"][0]["message"]["content"]); print();
print("finish_reason:", d["choices"][0]["finish_reason"], "| usage:", d["usage"])'
```

원시 HTTP로 chat completion을 한 번 요청한다.
max_tokens=64 제한에 걸려 finish_reason: length로 끊긴다.
이 답 텍스트를 4단계 결과와 비교할 기준으로 기억해두자.

물론입니다. 모델 서빙은 모델의 학습 결과를 공개하는 과정을 말합니다. 이 과정은 모델의 성능을 평가하고, 새로운 데이터를 사용하여 모델을 개선하는 데 도움이 됩니다. 이

finish_reason: length | usage: {'prompt_tokens': 42, 'total_tokens': 106, 'completion_tokens': 64, 'prompt_tokens_details': None}

4단계 — OpenAI SDK 스트리밍 chat completion (temperature=0)

```
# openai SDK용 격리 venv 준비 (최초 1회)
[ -d .venv ] || uv venv --python 3.11 .venv
.venv/bin/python3 -c "import openai" 2>/dev/null || uv pip install --python .venv/bin/python3 'openai>=1.60,<3'
```

같은 질문을 이번엔 OpenAI SDK로 stream=True 요청 — 토큰이 하나씩 도착

```
export OPENAI_BASE_URL="http://127.0.0.1:8000/v1"
```

```
.venv/bin/python3 client.py
```

client.py는 실제 OpenAI SDK로 stream=True 요청을 보내 토큰을 하나씩 받아 출력한다.

3단계 curl 결과와 완전히 같은 텍스트가 나오면, "curl과 SDK가 다른 답을 낸다"는 흔한 오해가

실은 샘플링 파라미터 미지정 탓이었고, temperature=0으로 고정하면 클라이언트·스트리밍 여부와 무관하게

같은 답이 나온다는 걸 확인한 것이다.

물론입니다. 모델 서빙은 모델의 학습 결과를 공개하는 과정을 말합니다. 이 과정은 모델의 성능을 평가하고, 새로운 데이터를 사용하여 모델을 개선하는 데 도움이 됩니다. 이

![3단계 curl 비-스트리밍 응답과 4단계 OpenAI SDK 스트리밍 응답이 "물론입니다. 모델 서빙은 ..."으로 완전히 동일하게 출력되고, curl 쪽에 finish_reason length:completion_tokens 64가 함께 표시된 실제 macOS 터미널 화면]

[image_followup/follow_02_curl_vs_sdk.png](#)

```
### 3단계 - curl 비-스트리밍 chat completion (temperature=0) ###
물론입니다. 모델 서빙은 모델의 학습 결과를 공개하는 과정을 말합니다. 이 과정은 모델의 성능을 평가하고, 새로운 데이터를
사용하여 모델을 개선하는 데 도움이 됩니다. 이

finish_reason: length | usage: {'prompt_tokens': 42, 'total_tokens': 106, 'completion_tokens': 64, 'prompt_tokens_detail
s': None}

### 4단계 - OpenAI SDK 스트리밍 (client.py, temperature=0) ###
물론입니다. 모델 서빙은 모델의 학습 결과를 공개하는 과정을 말합니다. 이 과정은 모델의 성능을 평가하고, 새로운 데이터를
사용하여 모델을 개선하는 데 도움이 됩니다. 이

### -> curl과 SDK 결과가 완전히 동일하면 정상 (temperature=0 재현성) ###
→ ~ □
```

5단계 — 정리

`docker compose down` # 컨테이너 종료 (VLLM_API_KEY가 export돼 있어야 함)

모델 캐시 볼륨(hf-cache-cpu)은 남겨둬 다음 실행이 빨라진다.

캐시까지 지우려면: `docker compose down -v`

[compose.yaml](#)

services:

vllm-cpu:

Apple GPU/Metal이 아니라 Docker Linux VM의 ARM CPU를 쓴다.

NVIDIA GPU 없이도 OpenAI 호환 API·continuous batching·KV Cache 동작을 그대로 확인할 수 있는 가장 이식성 높은 경로.

image: vllm/vllm-openai-cpu:latest-arm64

platform: linux/arm64

command:

```
- ${MODEL_ID:-Qwen/Qwen2.5-0.5B-Instruct}
- --served-model-name=${SERVED_MODEL_NAME:-assistant}
- --dtype=float16
- --max-model-len=${MAX_MODEL_LEN:-1024}
- --max-num-seqs=${MAX_NUM_SEQS:-4}
- --api-key=${VLLM_API_KEY:?set VLLM_API_KEY}
- --generation-config=vllm
```

ports: ["8000:8000"]

environment:

HF_TOKEN: \${HF_TOKEN:-}

vLLM CPU backend가 KV Cache에 예약할 호스트 메모리. 단위는 GiB다.

VLLM_CPU_KVCACHE_SPACE: \${VLLM_CPU_KVCACHE_SPACE:-1}

VLLM_CPU_OMP_THREADS_BIND: \${VLLM_CPU_OMP_THREADS_BIND:-0-7}

volumes:

```
- hf-cache-cpu:/root/.cache/huggingface
```

shm_size: 2gb

init: true

stop_grace_period: 30s

cap_add: [SYS_NICE]

```

    security_opt: [seccomp=unconfined]

volumes:
    # 최초 1회만 모델을 내려받고(Qwen2.5-0.5B-Instruct 기준 약 77초), 이후로는 이
    # 볼륨에 캐시된 가중치를 재사용해 기동이 빨라진다.
    hf-cache-cpu: {}

client.py
import os
from openai import OpenAI
client = OpenAI(
    base_url=os.getenv("OPENAI_BASE_URL", "http://127.0.0.1:8000/v1"),
    api_key=os.environ["VLLM_API_KEY"],
    timeout=30,
    max_retries=2,
)
stream = client.chat.completions.create(
    model=os.getenv("SERVED_MODEL_NAME", "assistant"),
    messages=[{"role": "user", "content": "모델 서빙을 한 문장으로 설명해줘."}],
    max_tokens=64,
    temperature=0,                                # greedy decoding
                                                    # 지정하지 않으면 vLLM 기본 샘플링(temperature=1.0류)이 적용돼,
                                                    # 같은 컨테이너 안에서 몇 번째 추론 요청이냐에 따라 결과가 크게 흔들린다.
                                                    # (실제로 관찰됨: README "원인 분석" 참고). 데모의 재현성을 위해 고정.

    stream=True,
)
for chunk in stream:
    # stream=True일 때는 매 chunk가 전체 응답이 아니라 델타(증분) 토큰만 담고 있음
    print(chunk.choices[0].delta.content or "", end="", flush=True)
print()

3단계 curl 결과와 4단계 SDK 결과가 완전히 같은 텍스트로 나오면,
"curl과 SDK가 다른 답을 낸다"는 혼란 오해가 실은 샘플링 파라미터 미지정 탓이었고,
temperature=0으로 고정하면 클라이언트·스트리밍 여부와 무관하게 같은 답이 나온다는 것을 확인한 것이다.

```



수고하셨습니다.

본 문서는 SK AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

